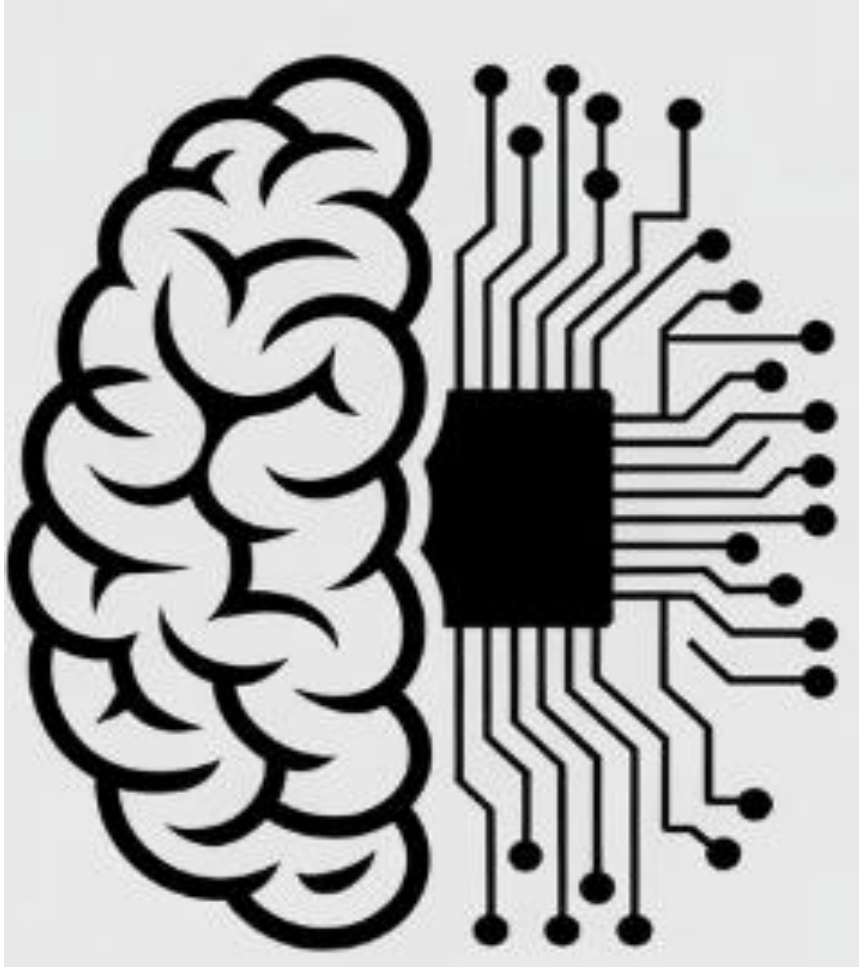
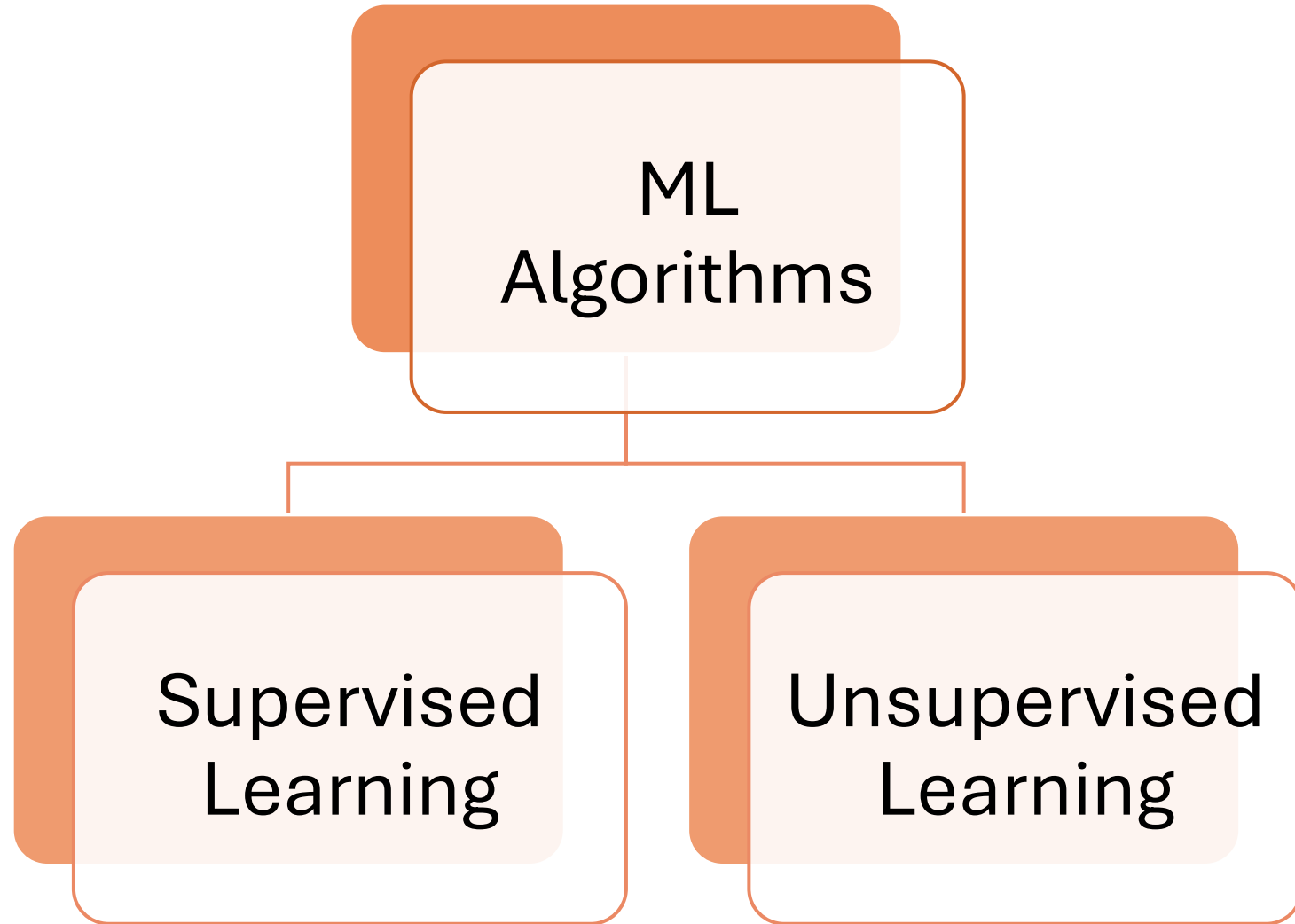




Clustering

Dr. Sobia Tariq Javed

Categories of Machine Learning Algorithms



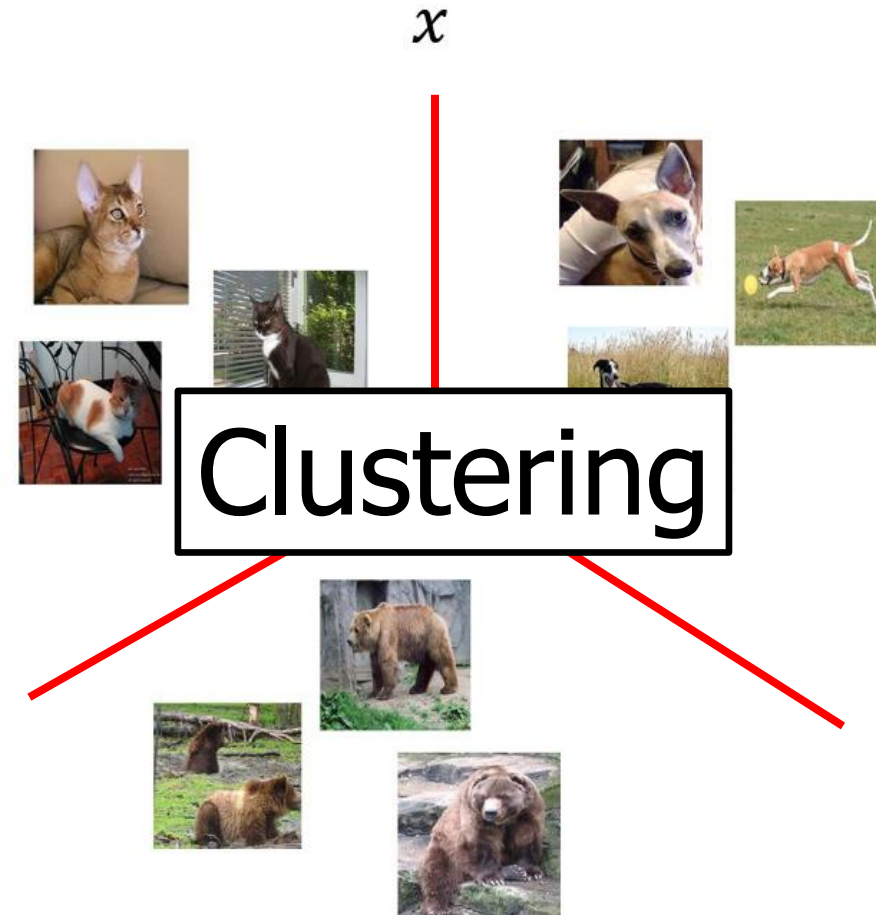
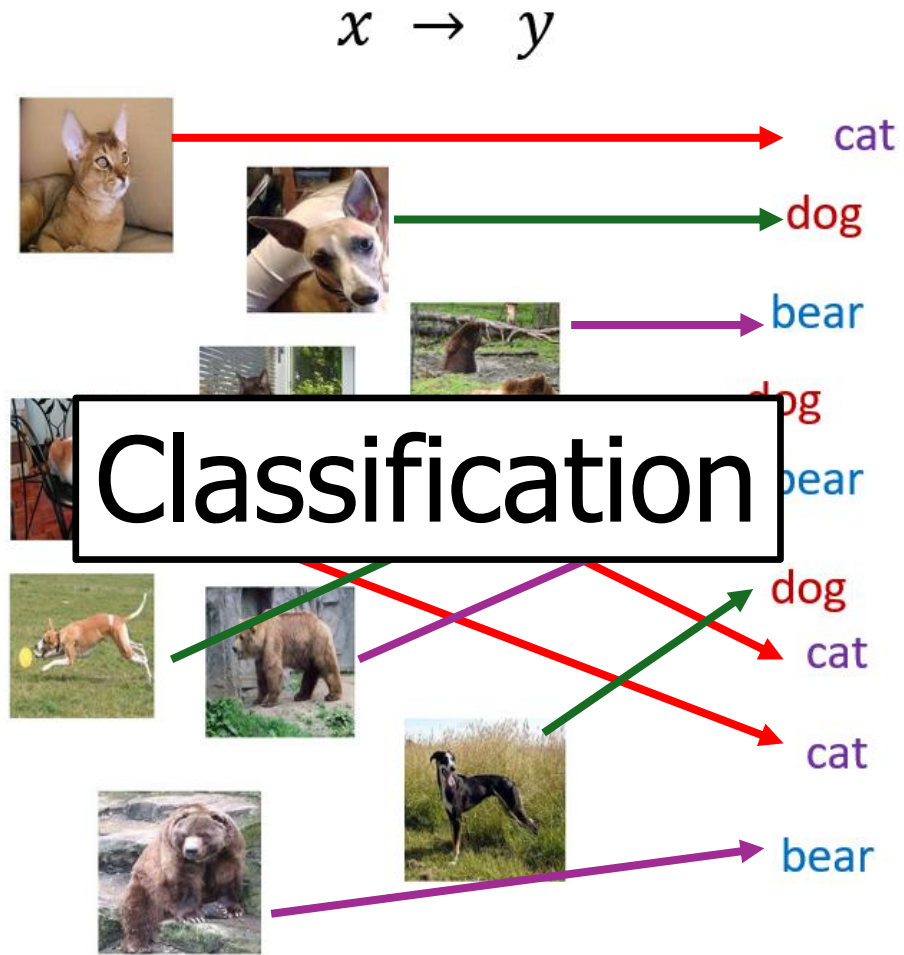
Supervised Learning

- Learning from labeled data
- **Input-output** pairs are provided during training
- **Objective:** learn a mapping function from features to target labels
- **Common tasks:**
 - Classification
 - Regression

Unsupervised Learning

- Learning from unlabeled data
- No predefined target variable
- **Objective:** discover hidden structure or patterns in data
- **Common tasks:**
 - Clustering
 - Dimensionality Reduction
 - Association Rule Mining

Supervised Learning vs Unsupervised Learning

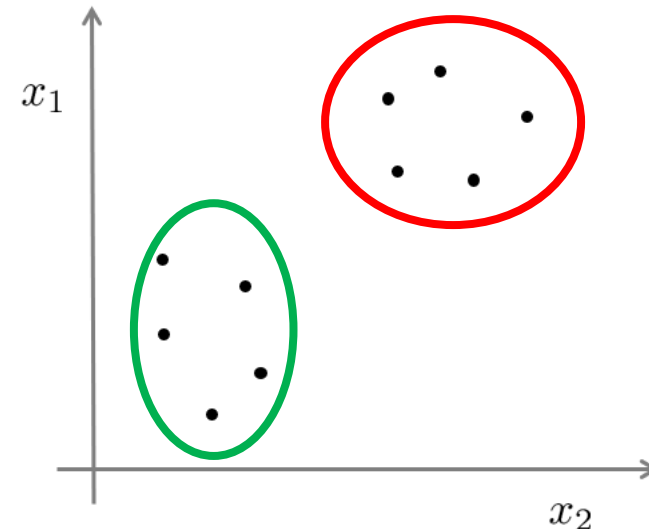


Clustering

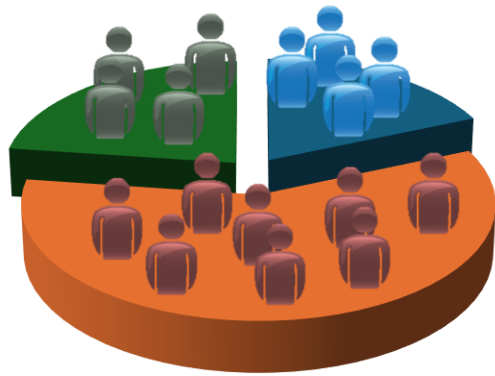
- It is an **unsupervised learning technique** used to group similar data points into **clusters** based on **shared patterns or features**, typically using a defined **distance or similarity measure**.
- Put data points that are similar into the same group, and different points into different groups

- **Training set:**

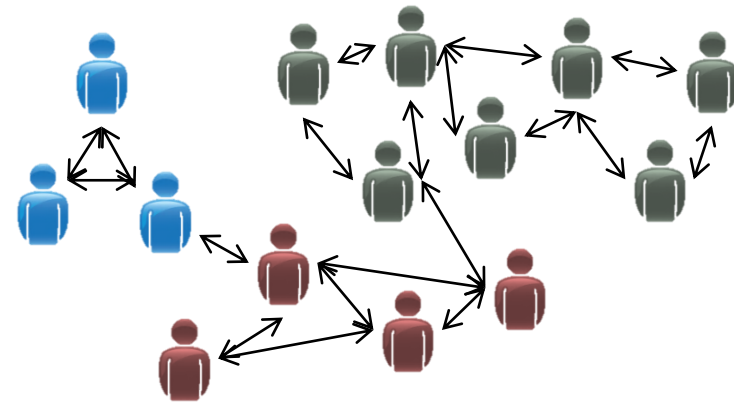
$$\{x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)}\}$$



Application of Clustering



Market segmentation



Social network analysis



Organize computing clusters

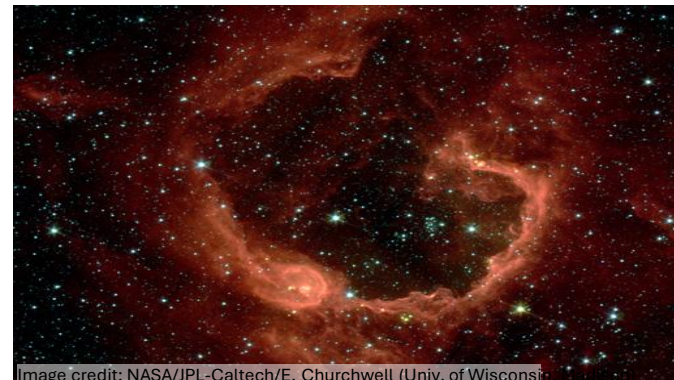


Image credit: NASA/JPL-Caltech/E. Churchwell (Univ. of Wisconsin)

Astronomical data analysis

Types of Clustering

- Clustering techniques can be classified into several major types based on how they form groups from data.
 - **Partitioning Clustering**
 - **Hierarchical Clustering**

1) Partitioning Clustering

- Divides data into a **fixed number of non-overlapping clusters**, where each data point belongs to only **one cluster**.
- **Example:**
 - K-Means
 - K-Medoids
- **Best for:**
 - Large datasets
 - Clearly separated clusters
- **Limitation:**
 - Need to predefine number of clusters (K)

2) Hierarchical Clustering

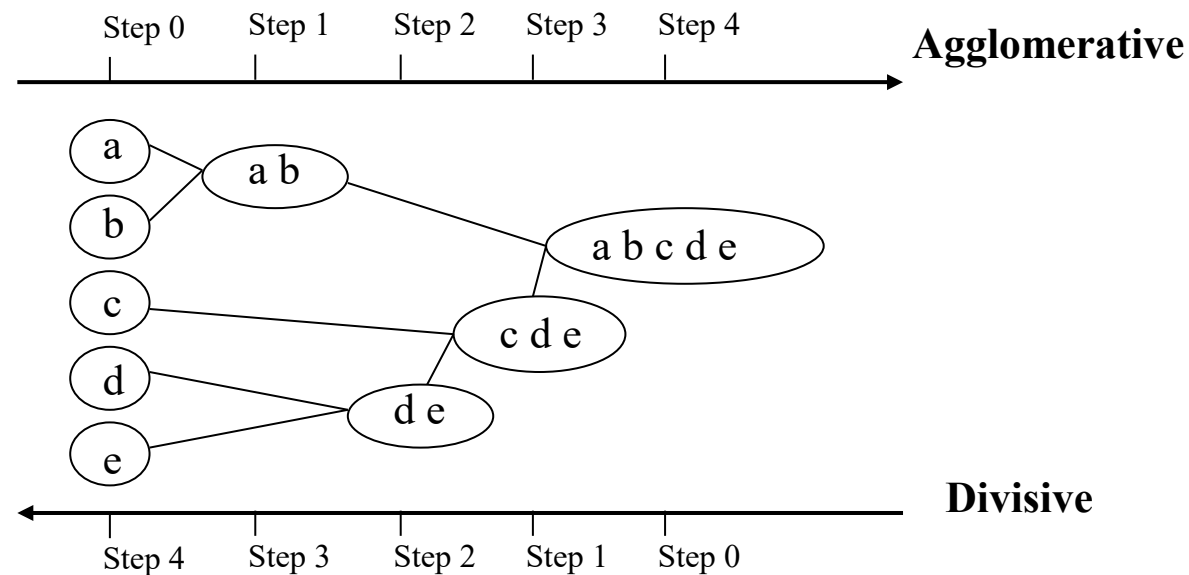
- Creates a tree-like structure of clusters called a dendrogram.
- **Types:**
 - (a) Agglomerative (Bottom-Up)**
 - Start with each point as its own cluster
 - Merge similar clusters step by step
 - (b) Divisive (Top-Down)**
 - Start with one large cluster
 - Split into smaller clusters

2) Hierarchical Clustering

- **Best for:**
 - Understanding cluster relationships
- **Limitation:**
 - Computationally expensive for large datasets

Agglomerative vs. Divisive

- Agglomerative and divisive clustering on the data set {a, b, c, d, e}



Common Distance measures:

- Distance measure will determine how the **similarity of two elements** is calculated and it will influence the **shape of the clusters**.

Euclidean Distance (Most Common)

- Measures straight-line distance between two points.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- **Best for:**
 - Continuous numerical data
 - K-Means clustering

Manhattan Distance (City Block Distance)

- Measures distance by summing absolute differences.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

- **Best for:**
 - Grid-like paths
 - High-dimensional data in some cases

Minkowski Distance

- Generalized form of Euclidean and Manhattan.

$$d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}$$

- **Special cases:**
 - $p=1 \rightarrow$ Manhattan
 - $p=2 \rightarrow$ Euclidean
- **Best for:**
 - Flexible distance modeling

Cosine Similarity / Cosine Distance

- Measures angle between vectors rather than physical distance.

$$\text{Cosine Similarity} = \frac{x \cdot y}{\|x\| \|y\|}$$

- **Best for:**
 - Text mining
 - Document clustering
 - High-dimensional sparse data

Hamming Distance

- Counts number of differing positions between categorical or binary vectors.
- **Example:**
 - 10101 vs 11100 → distance = 2
- **Best for:**
 - Binary attributes
 - Categorical data

Jaccard Distance

- Measures dissimilarity between sets.

$$\text{Jaccard Distance} = 1 - \frac{|A \cap B|}{|A \cup B|}$$

- **Best for:**
 - Binary data
 - Set comparison
 - Market basket analysis

K-Means Clustering

- A **partition-based clustering** approach that divides data into **distinct clusters**.
- Each **cluster** is associated with a **centroid** (central point).
- Each **data point** is assigned to the cluster with the **nearest centroid**.
- The **number of clusters (K)** must be specified in advance.

K-Means Clustering



The basic algorithm is **simple and computationally efficient**.

-
- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change
-

K-Mean Clustering

- **Initial centroids** are usually selected **randomly**, so results may **differ** across runs.
- The **number of clusters (K)** must be specified in advance.
- Each cluster is represented by a **centroid**, typically the mean of its assigned points.

K-Mean Clustering

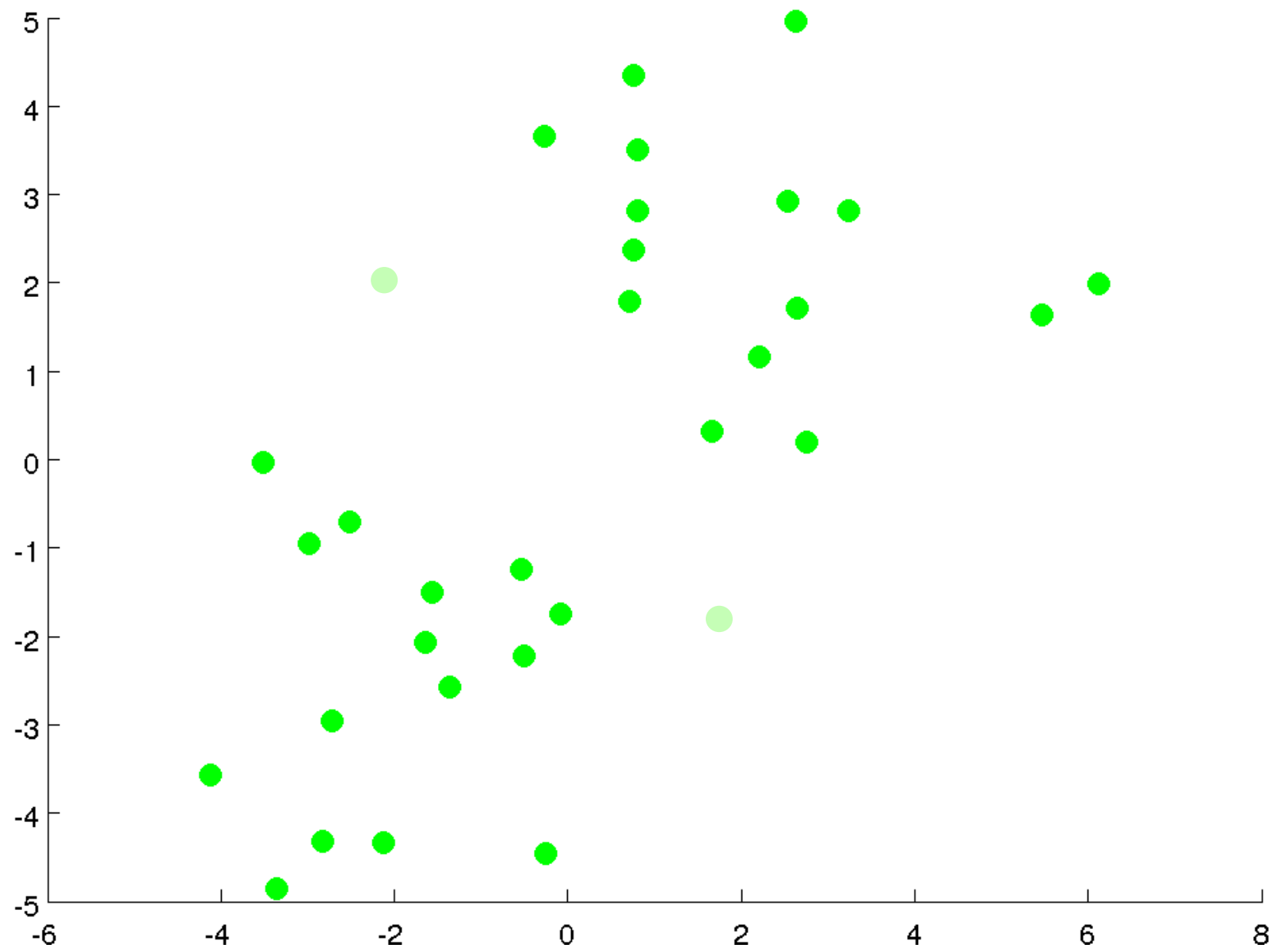
- **Closeness** is commonly measured using **Euclidean distance, cosine similarity, correlation, or similar metrics.**
- K-Means iteratively assigns points to the **nearest centroid** and **updates centroids** to minimize **within-cluster variance.**

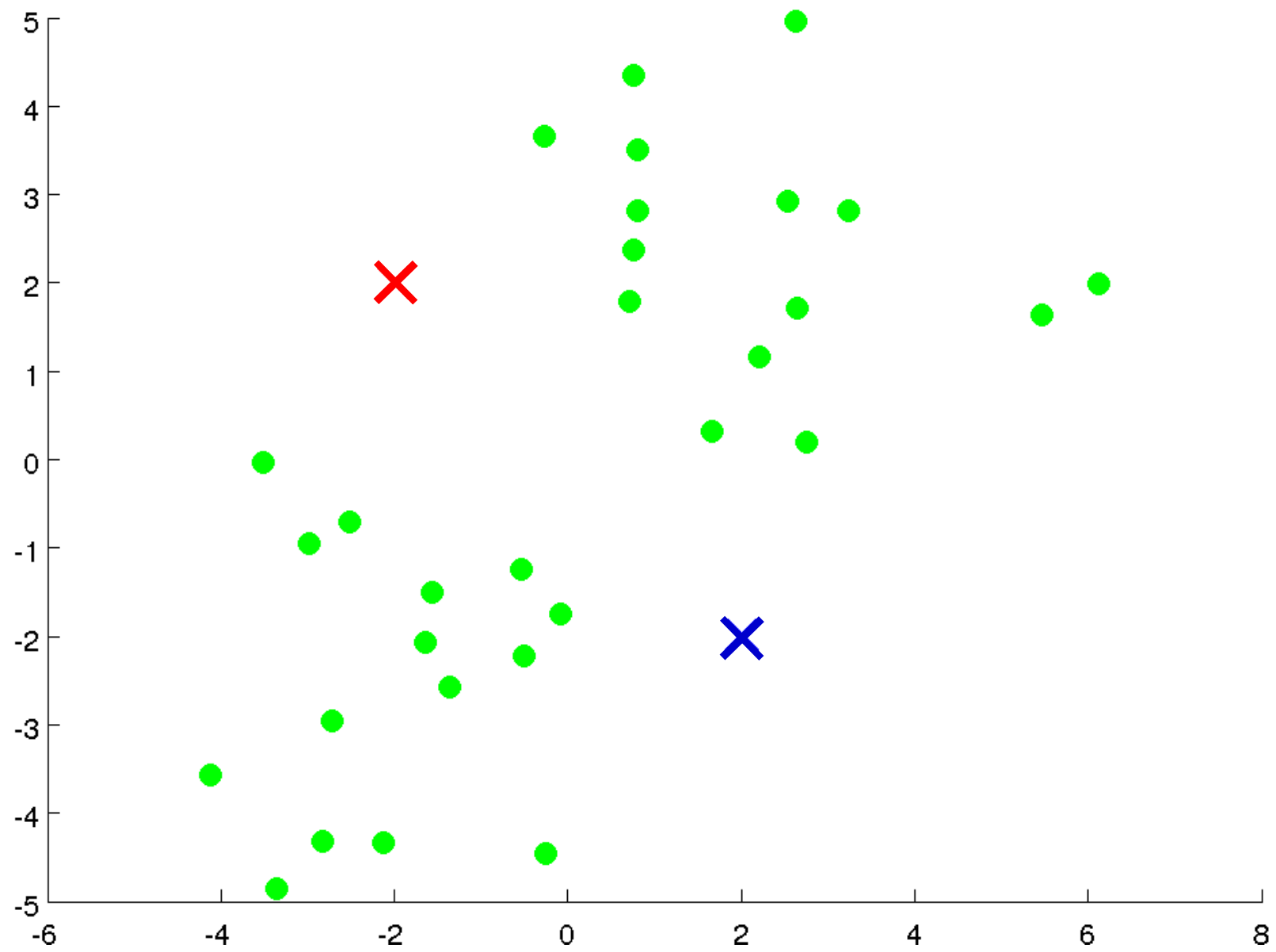
K-Mean Clustering

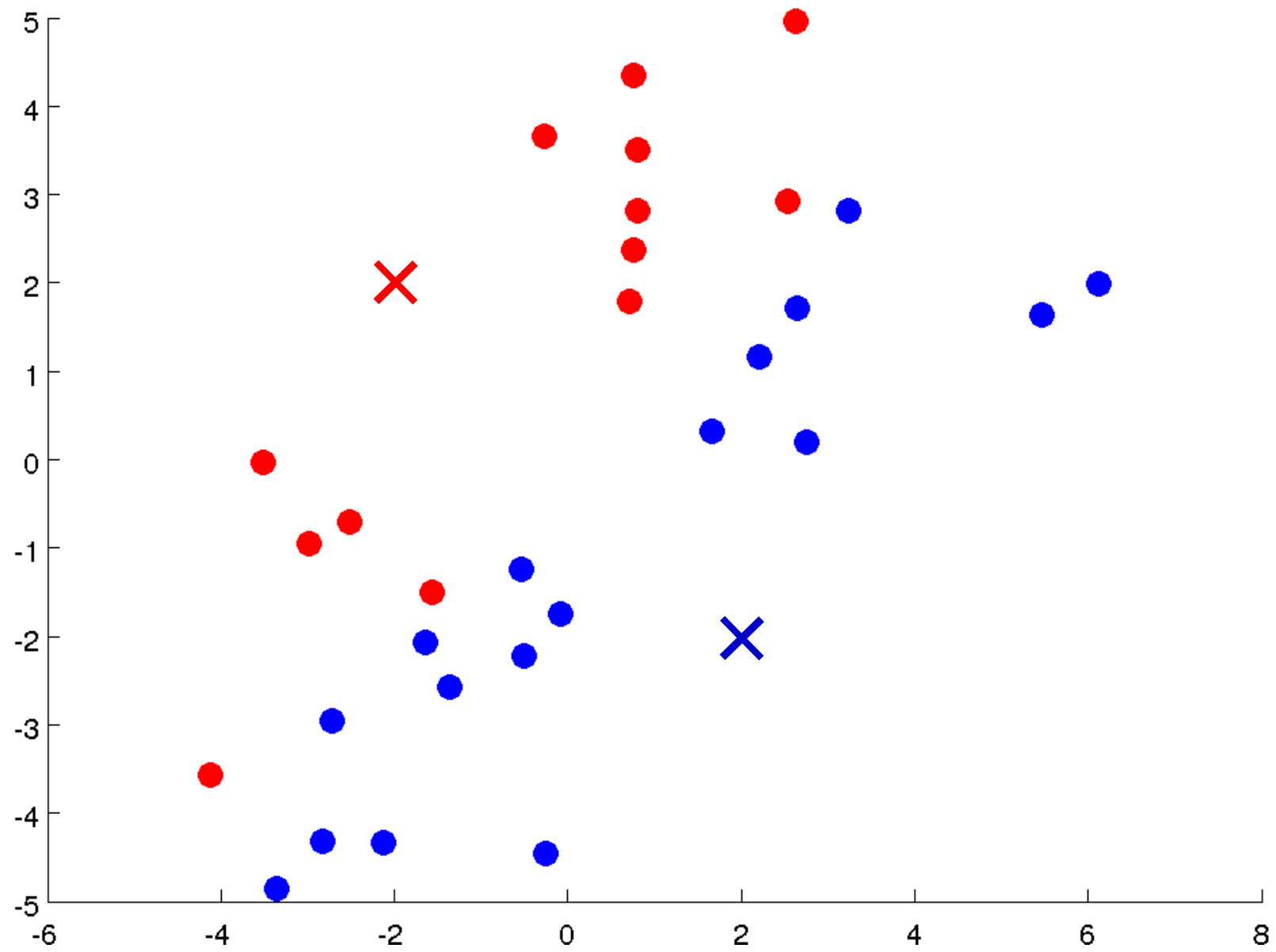
- **Most convergence** typically occurs within the **first few iterations**.
- The algorithm generally **converges**, but may settle at a **local optimum** depending on **initialization**.
- **Stopping** usually occurs when **centroid movement** is minimal or very few points change clusters.

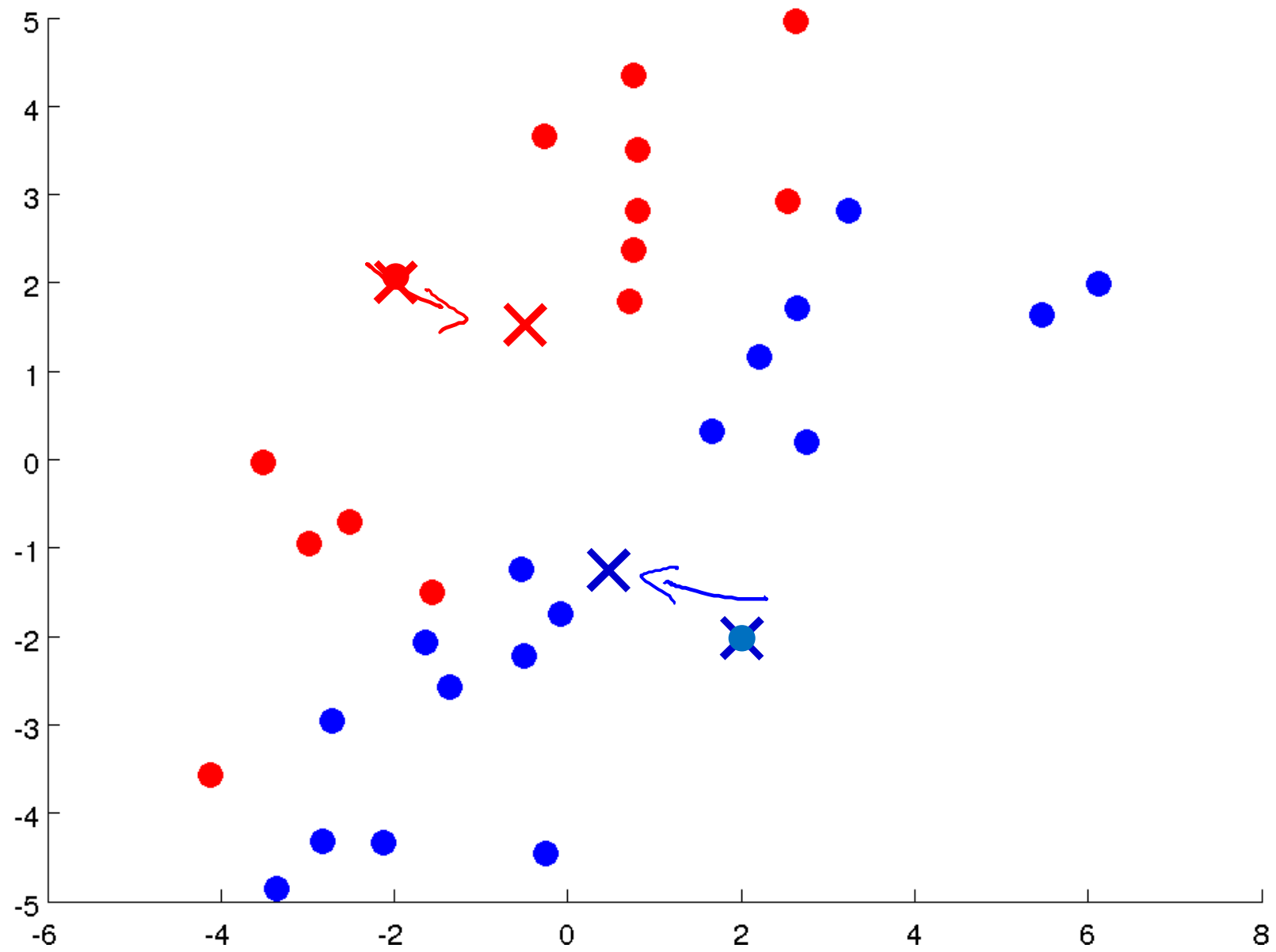
K-Mean Clustering

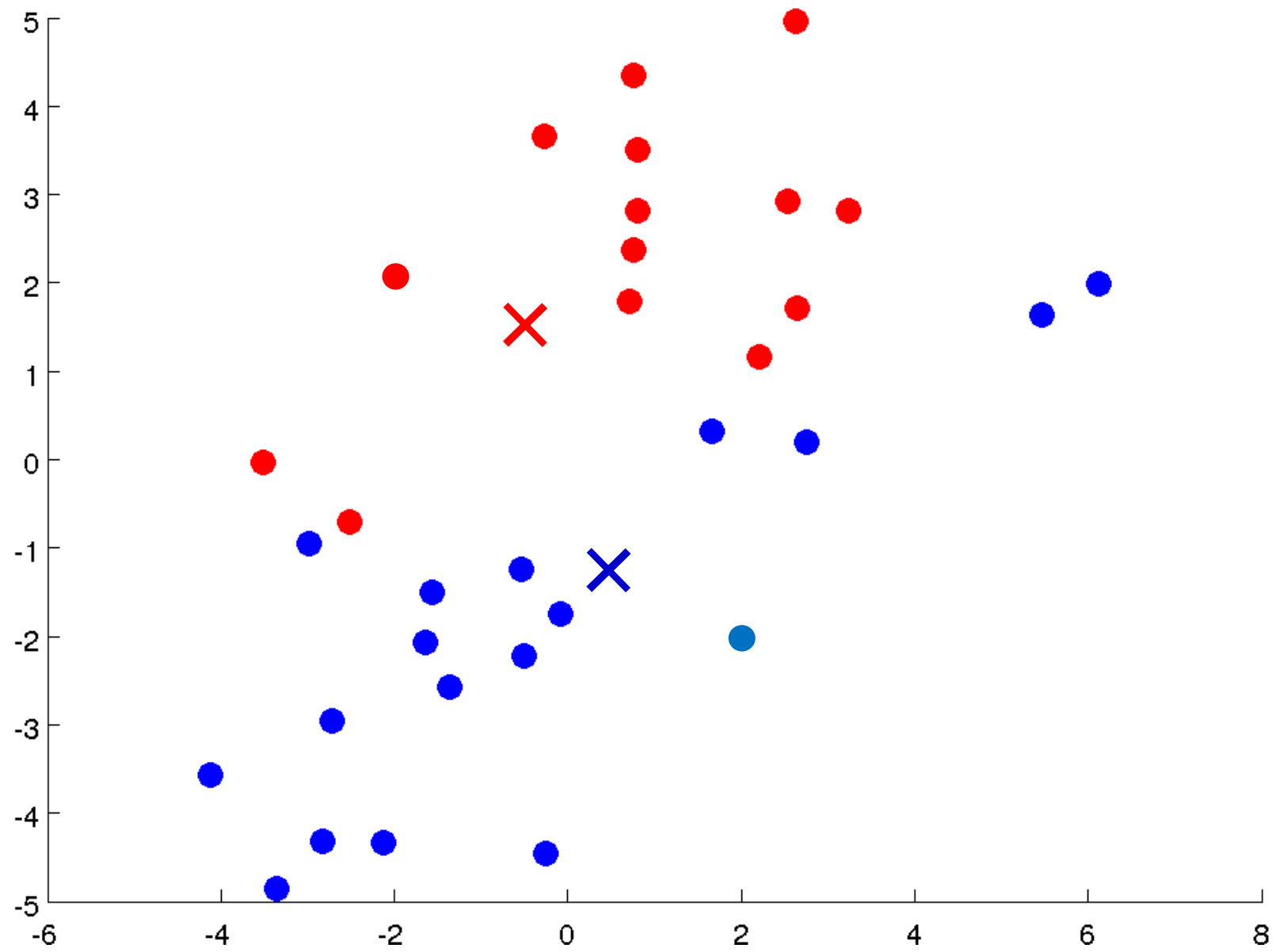
- **Feature scaling** is often important for accurate distance-based clustering.
- K-Means works best for **compact, spherical, similarly sized clusters**, but is sensitive to **outliers and irregular shapes**.

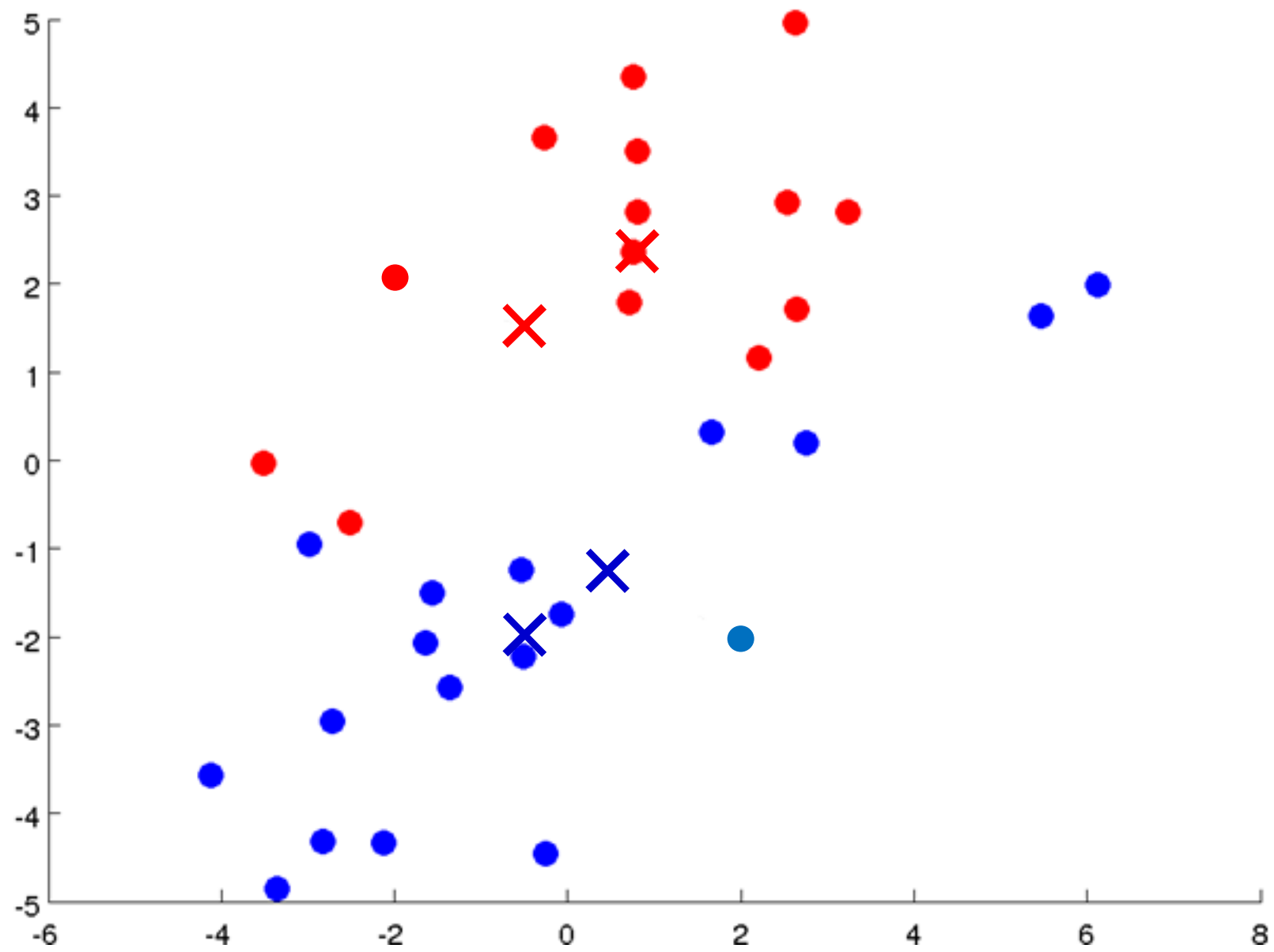


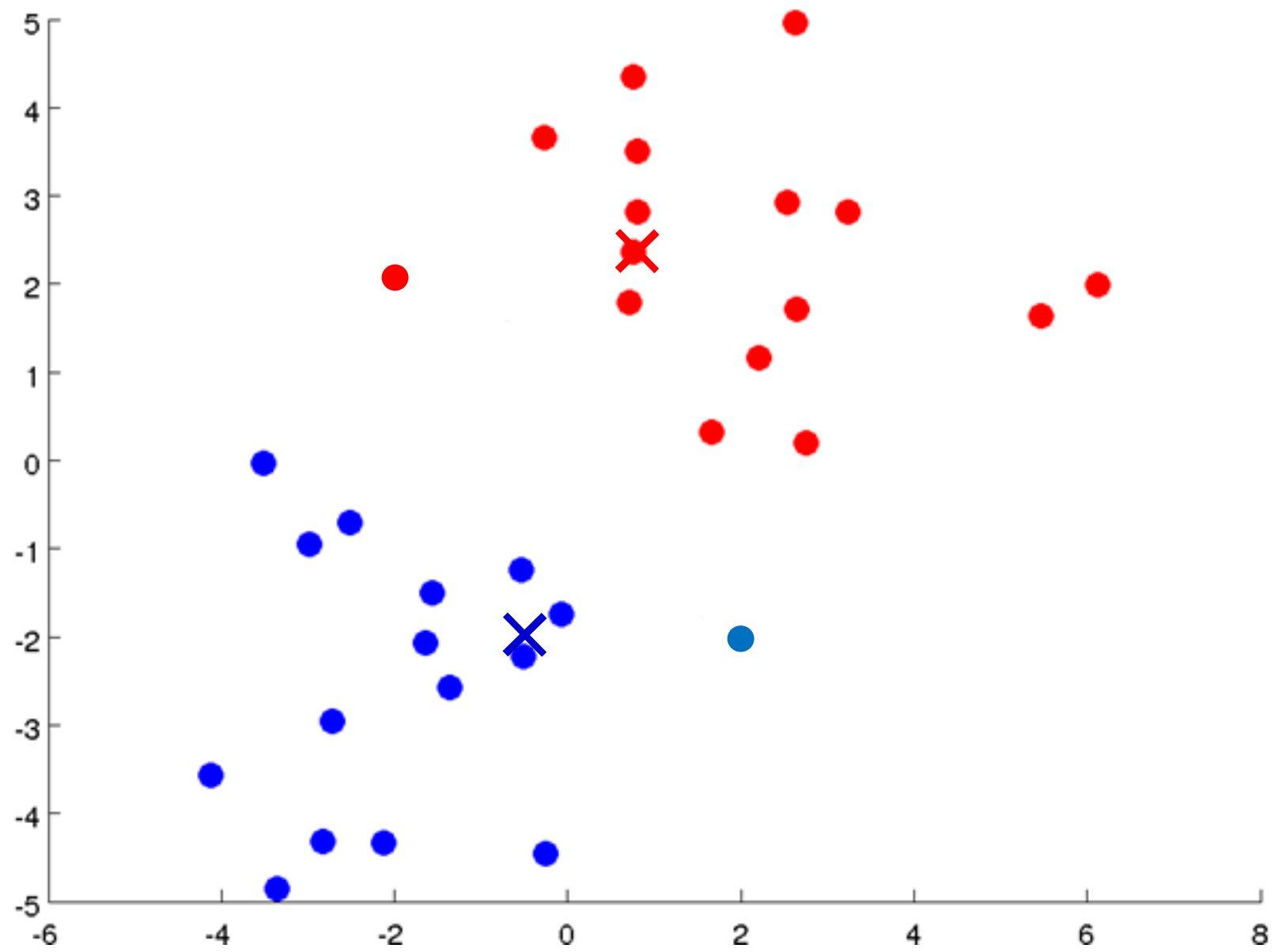


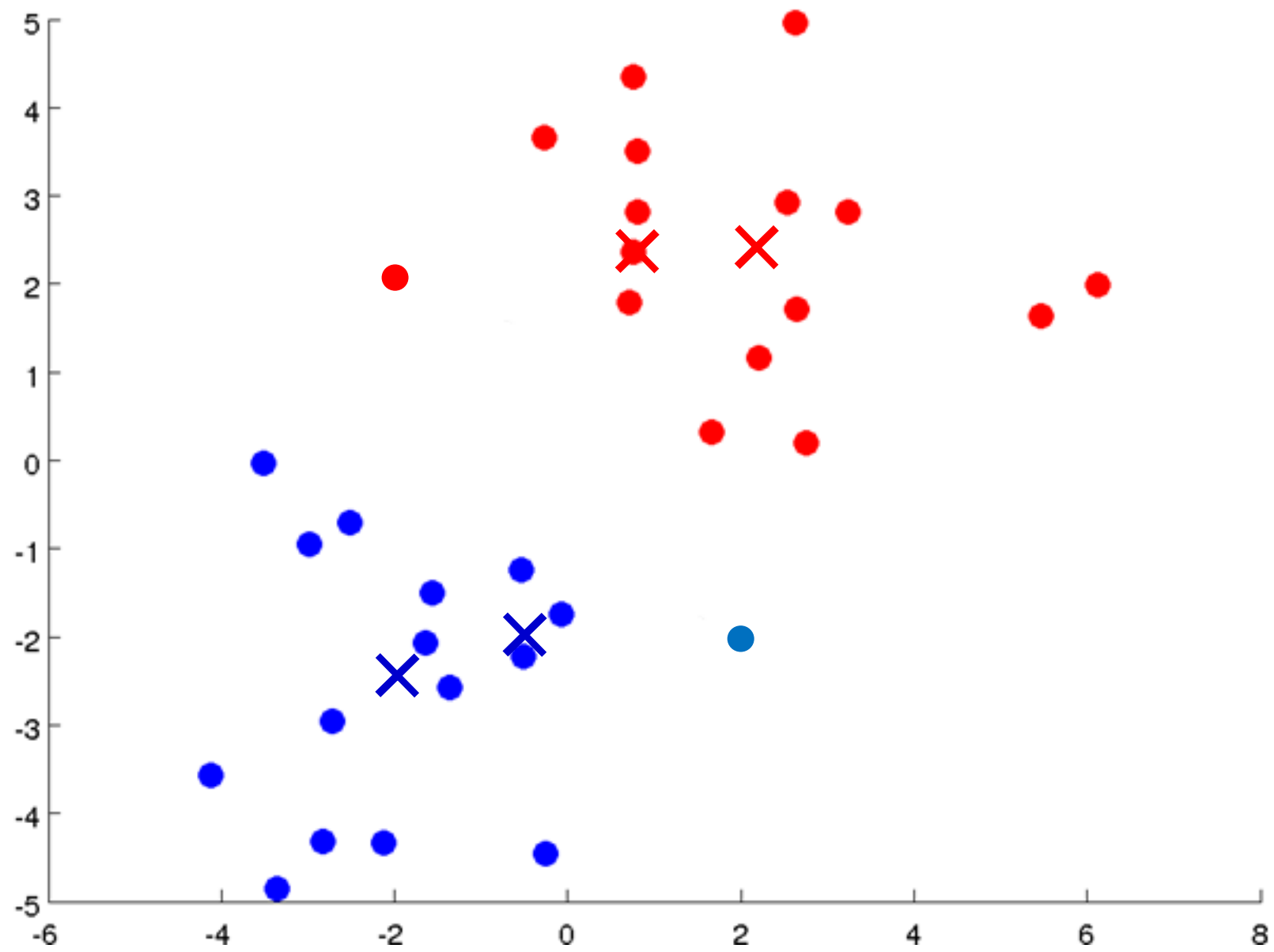


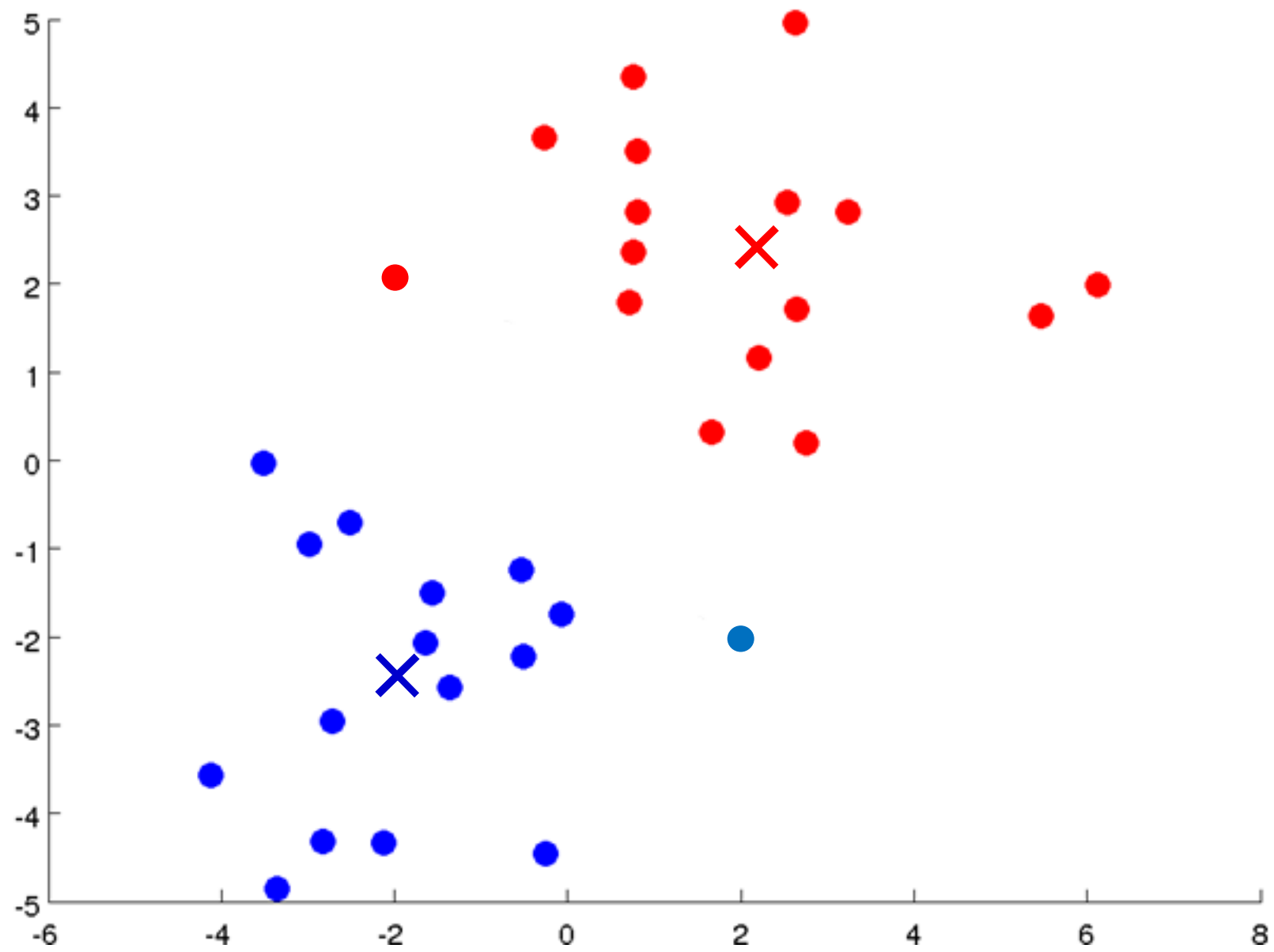












Exercise

- Consider the following seven points. Cluster using K-mean technique.

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Exercise

- **Step 1:**
- **Initialization:**
 - Randomly we choose following two centroids ($k=2$) for two clusters.
 - In this case the 2 centroid are:
 $m1=(1.0,1.0)$ and $m2=(5.0,7.0)$.

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

	Individual	Mean Vector
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

Exercise

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Individual	Centroid 1	Centroid 2
1	0	7.21
2	1.12	6.10
3	3.61	3.61
4	7.21	0
5	4.72	2.5
6	5.31	2.06
7	4.30	2.92

- **Step 2:**
- Compute **distance** of each given point from the 2 centroids (1,1) and (5,7).
- Thus, we obtain two clusters containing: {1,2,3} and {4,5,6,7}.

$$d(m_1, 2) = \sqrt{|1.0 - 1.5|^2 + |1.0 - 2.0|^2} = 1.12$$

$$d(m_2, 2) = \sqrt{|5.0 - 1.5|^2 + |7.0 - 2.0|^2} = 6.10$$

- Their new centroids are:

$$m_1 = \left(\frac{1}{3}(1.0 + 1.5 + 3.0), \frac{1}{3}(1.0 + 2.0 + 4.0) \right) = (1.83, 2.33)$$

$$m_2 = \left(\frac{1}{4}(5.0 + 3.5 + 4.5 + 3.5), \frac{1}{4}(7.0 + 5.0 + 5.0 + 4.5) \right) \\ = (4.12, 5.38)$$

Exercise

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

$m1 = (1.83, 2.33)$, $m2 = (4.12, 5.38)$

- **Step 3:**
- Now using these centroids, we compute the Euclidean distance of each object, as shown in the table.
- Therefore, the new clusters are: {1,2} and {3,4,5,6,7}.
- Next centroids are: $m1 = (1.25, 1.5)$ and $m2 = (3.9, 5.1)$

Individual	Centroid 1	Centroid 2
1	1.57	5.38
2	0.47	4.28
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

Exercise

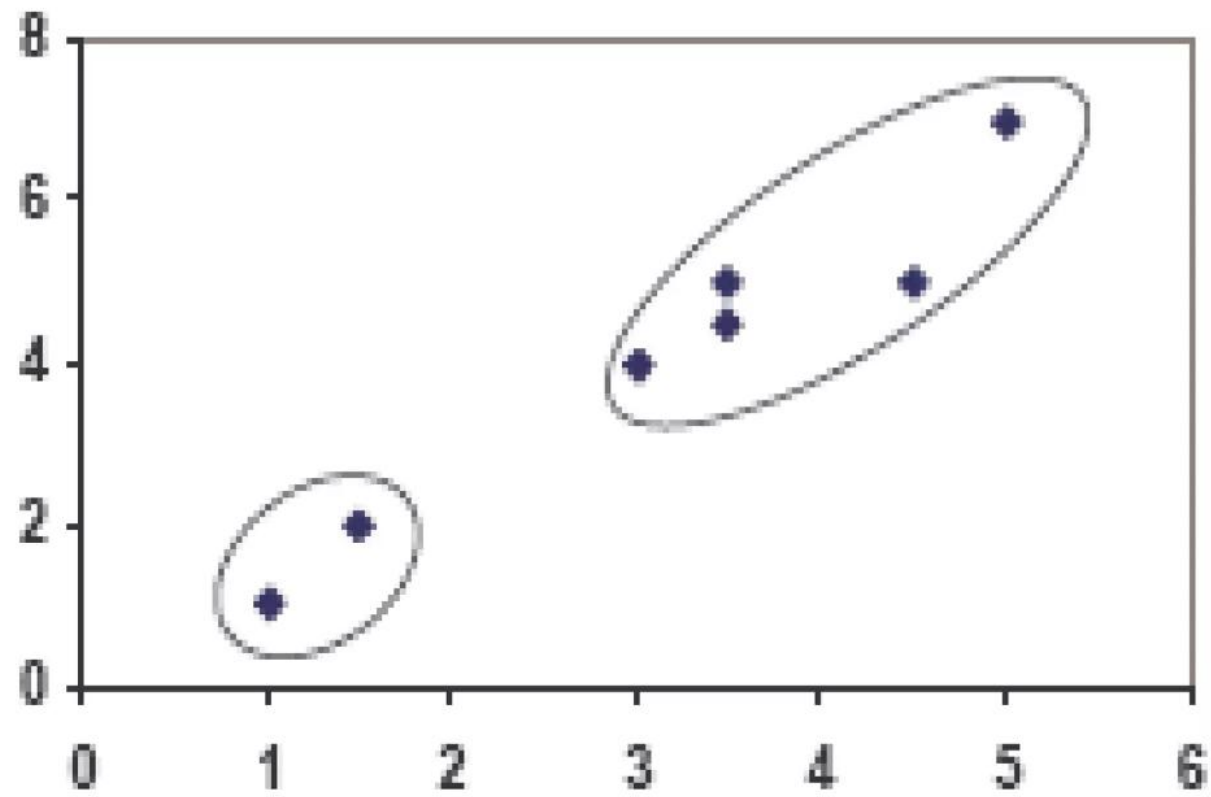
- **Step 4:**
- The clusters obtained are: {1,2} and {3,4,5,6,7}.
- Therefore, there is no change in the cluster.
- Thus, the algorithm comes to a halt here and the final result consists of 2 clusters {1,2} and {3,4,5,6,7}.

$m1=(1.25,1.5)$, $m2 = (3.9,5.1)$



Individual	Centroid 1	Centroid 2
1	1.57	5.38
2	0.47	4.28
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

Plot



Repeat the algorithm with $K=3$

- Step 1
- Clustering with initial centroids set to the first three points

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Point	M1 (Point 1)	M2 (Point 2)	M3 (Point 3)	Cluster
1	0	1.11	3.61	1
2	1.12	0	2.5	2
3	3.61	2.5	0	3
4	7.21	6.10	3.61	3
5	4.72	3.61	1.12	3
6	5.31	4.24	1.80	3
7	4.30	3.20	0.71	3

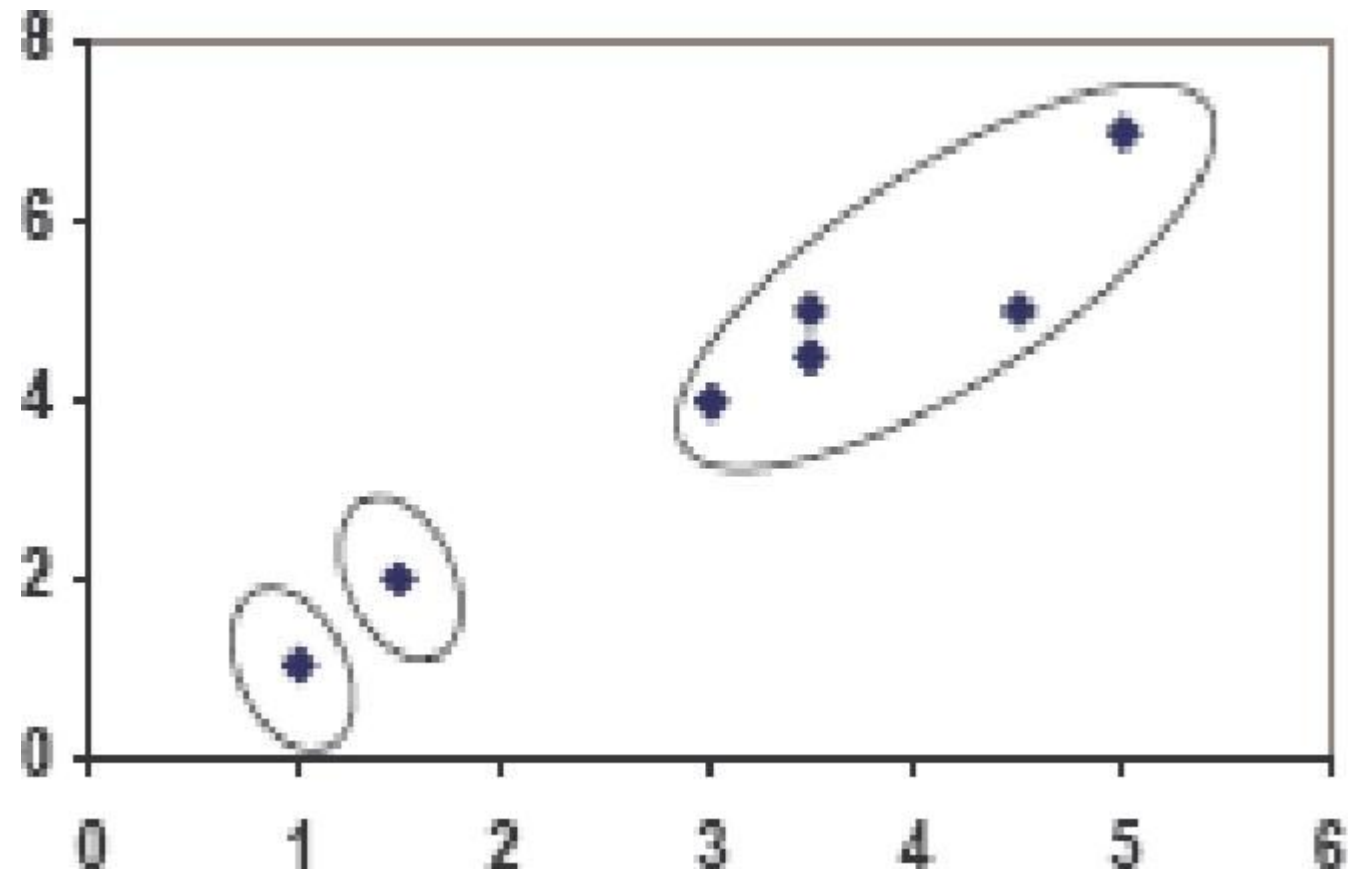
Repeat the algorithm with $K=3$

- **Step 2**

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Point	M1 (Point 1)	M2 (Point 2)	M3 (Point 3)	Cluster
1	0	1.11	3.61	1
2	1.12	0	2.5	2
3	3.61	2.5	0	3
4	7.21	6.10	3.61	3
5	4.72	3.61	1.12	3
6	5.31	4.24	1.80	3
7	4.30	3.20	0.71	3

Plot



Evaluating K-Means Clusters

- Most common measure is **Sum of Squared Error (SSE)**
- For each point, the **error** is the distance to the **nearest cluster**.
- To get **SSE**, we square these errors and sum them

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

- **x** is a data point in cluster **C_i** and **m_i** is the representative point (mean point) for cluster **C_i**

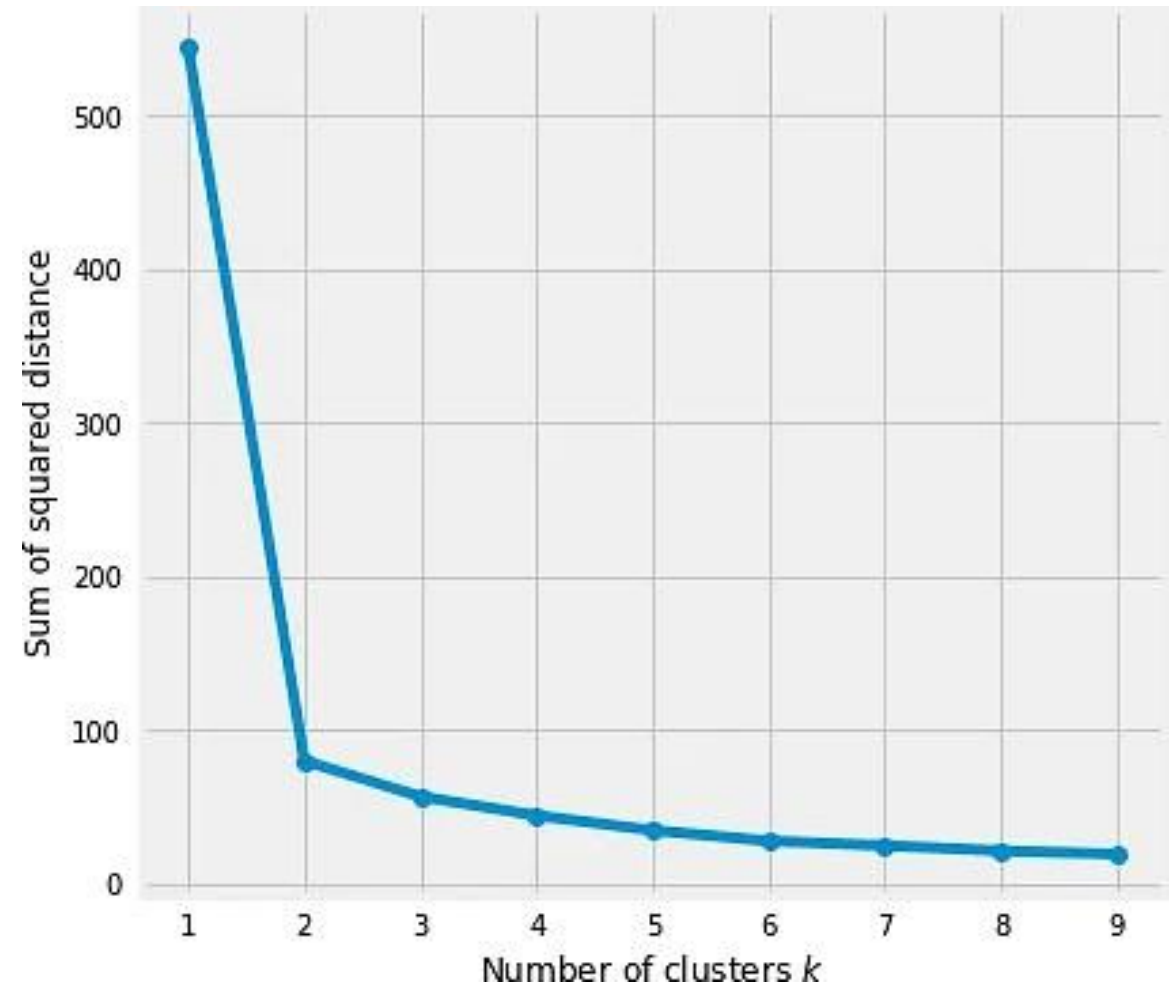
Given two clustering results (with different initial centroids), the one with the smaller error is preferred.

Evaluating K-Means Clusters

- One easy way to reduce **SSE** is to increase **K**, the number of clusters.
- A good cluster with **smaller K** can have a **higher SSE** than a poor clustering with **higher K**.

Selecting the Optimal K

- **Elbow method** gives us an idea on what a **good k number of clusters** would be based on the sum of squared distance (**SSE**) between data points and their assigned clusters' centroids.
- We pick k at the spot where SSE **starts to flatten out** and forming an **elbow**.



Strengths of K-Means Algorithm

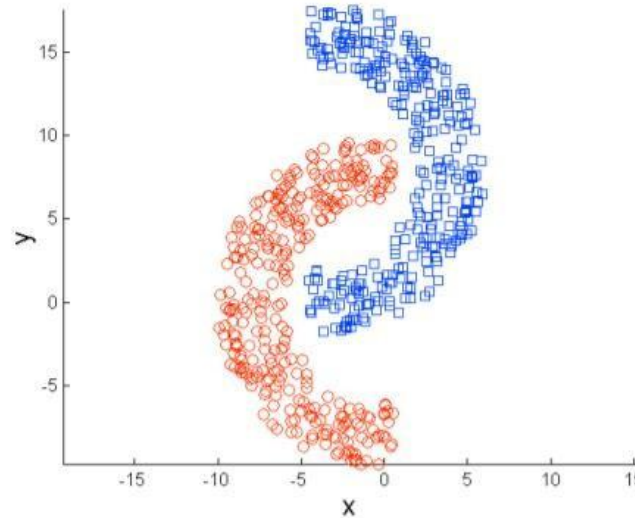
- Simple and easy to implement
- Fast and computationally efficient
- Scales well to large datasets
- Produces easy-to-interpret clusters (centroids)
- Works well for compact, well-separated (spherical) clusters
- Converges quickly in most cases

Limitations of K-Means Algorithm

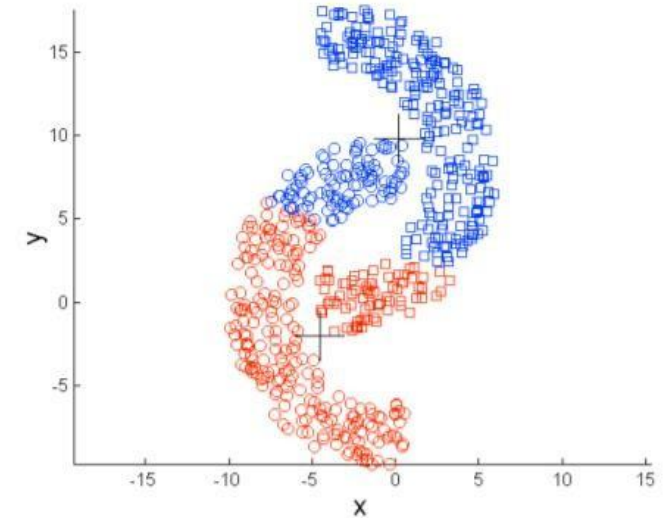
- Requires **number of clusters (K)** to be predefined
- **Sensitive** to **initial centroid selection**
- Can converge to a **local optimum**
- **Sensitive** to **outliers** and **noise**
- **Feature scaling** is necessary for reliable results
- May produce **empty clusters** in some cases

Limitations of K-Means Algorithm

- **Performs poorly with non-spherical, or uneven-density clusters.**
- **Non-spherical clusters (Non Globular Shape)**
 - Clusters that are not “round” in shape, such as:
 - crescent (moon-shaped)
 - spiral
 - elongated or curved structures



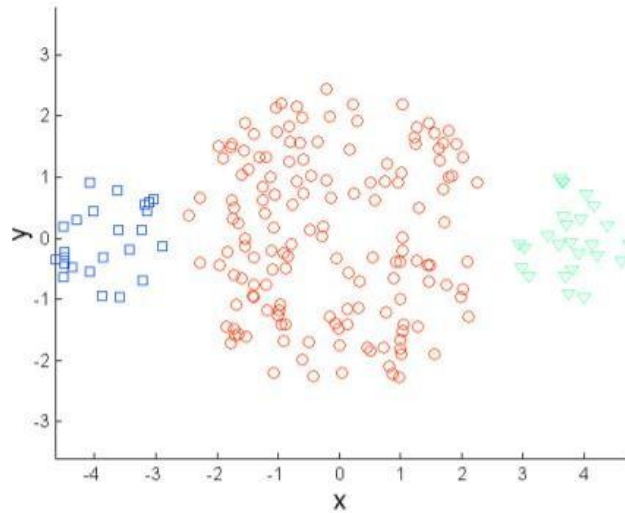
Original Points



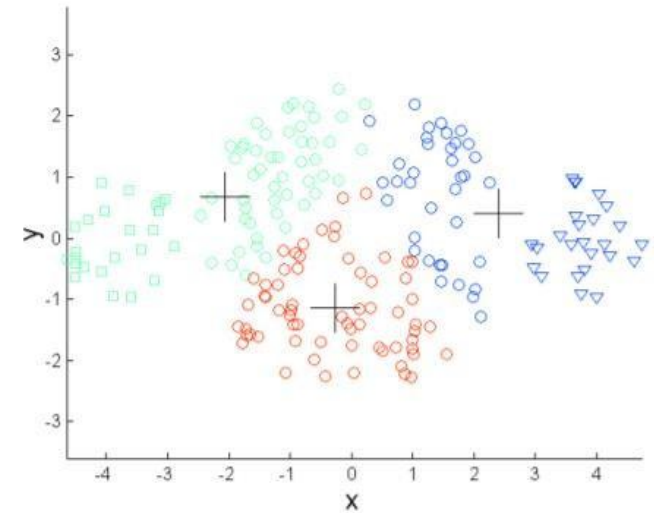
K-means (2 Clusters)

Limitations of K-Means Algorithm

- **Uneven-density clusters**
 - Clusters where:
 - One cluster is dense (points close together)
 - Another is sparse (points spread out)



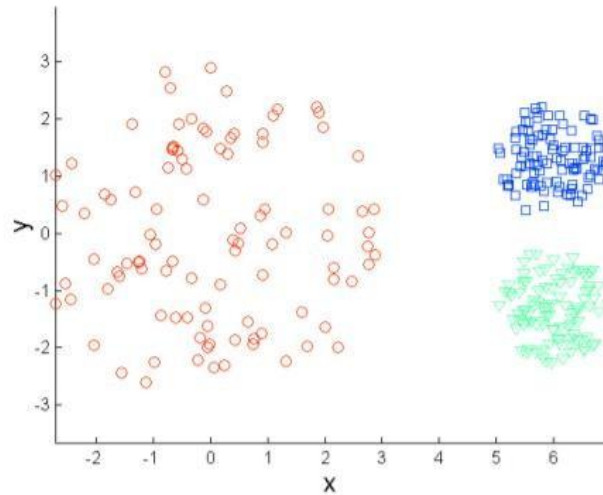
Original Points



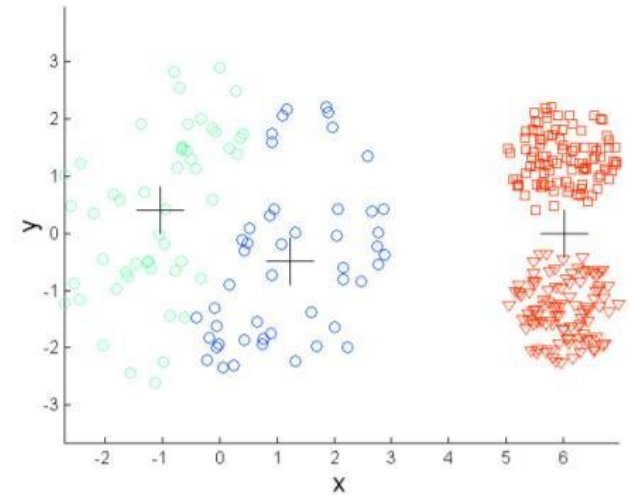
K-means (3 Clusters)

Limitations of K-Means Algorithm

- **Uneven-density clusters**
 - Clusters where:
 - One cluster is dense (points close together)
 - Another is sparse (points spread out)



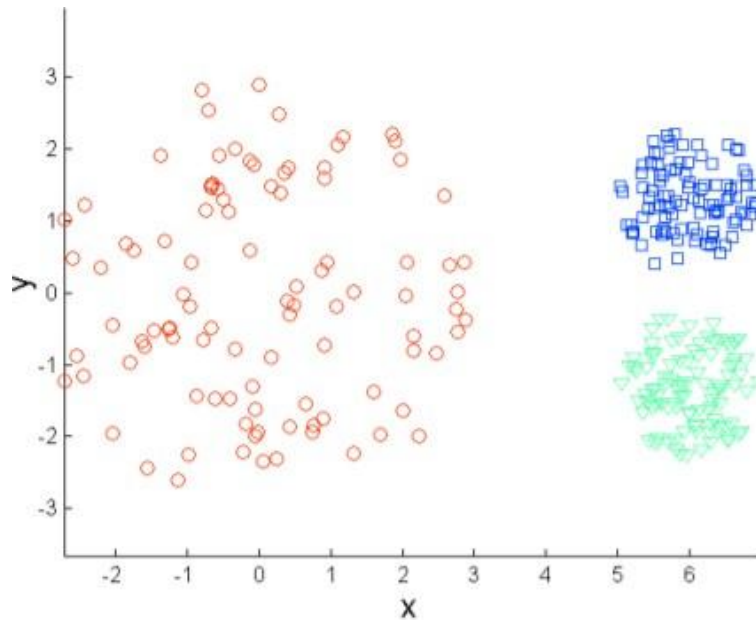
Original Points



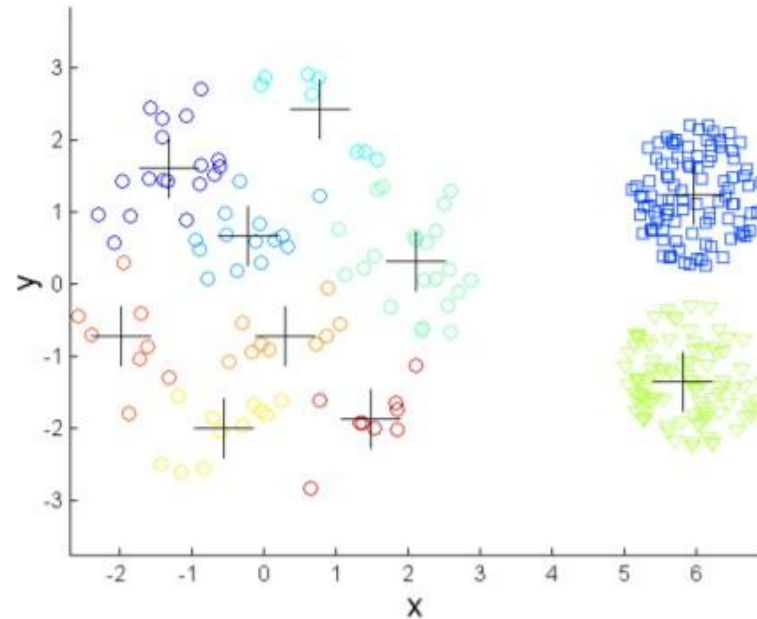
K-means (3 Clusters)

Solution

- One solution is to use many clusters. Find parts of clusters, but need to put together



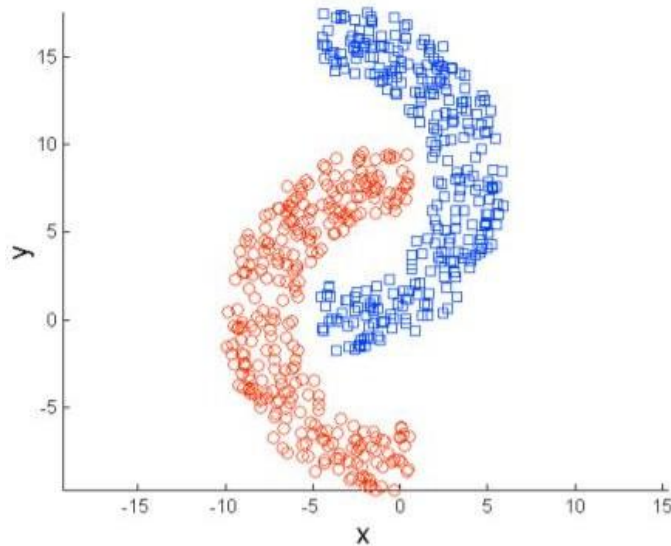
Original Points



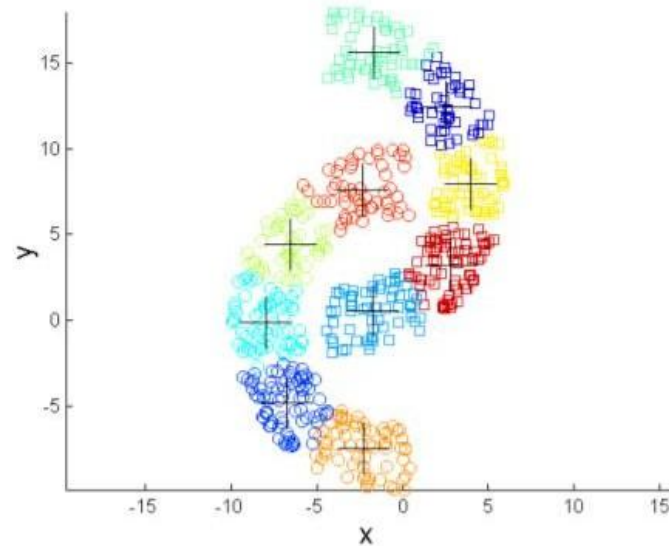
K-means Clusters

Solution

- One solution is to use many clusters. Find parts of clusters, but need to put together



Original Points



K-means Clusters

K-mediods Clustering

- Always data points are selected as **mediods**
- Less **sensitive to outliers**
- For **K-Means**, we were working with centroids.
 - The relationship between centroids and medoids is similar to the relationship between means and medians.
 - Medoids and medians will always be one of the observations in the data, while that's not necessarily the case for centroids and means.

- **PAM: Partitioning around Mediods**

Reading Homework:

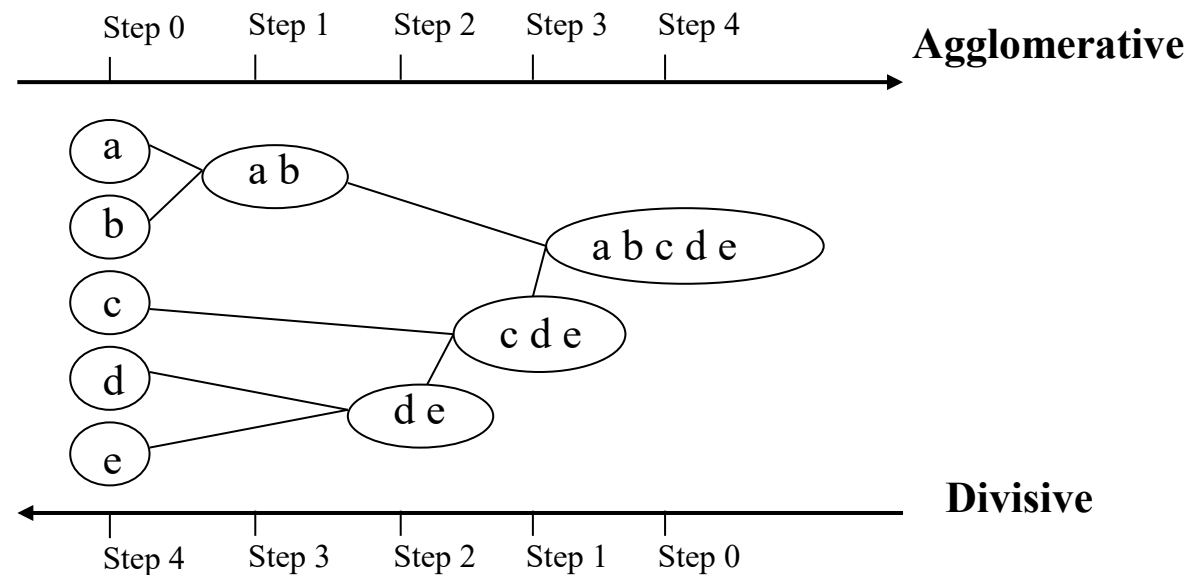
- <https://www.geeksforgeeks.org/ml-k-medoids-clustering-with-example/>
- <https://medium.com/@ali.soleymani.co/beyond-scikit-learn-is-it-time-to-retire-k-means-and-use-this-method-instead-b8eb9ca9079a>

Agglomerative Clustering

- It is a **hierarchical clustering method** that builds clusters **bottom-up**.
- **Main Idea:**
 - Begin by treating each data point as an **individual cluster**.
 - Calculate the **similarity or distance** between all **clusters**.
 - Merge the two **closest clusters** into a **single cluster**.
 - Recompute distances between the **newly formed cluster** and remaining clusters.
 - Repeat the merging process until the **desired number of clusters** is reached or all data points form one cluster.

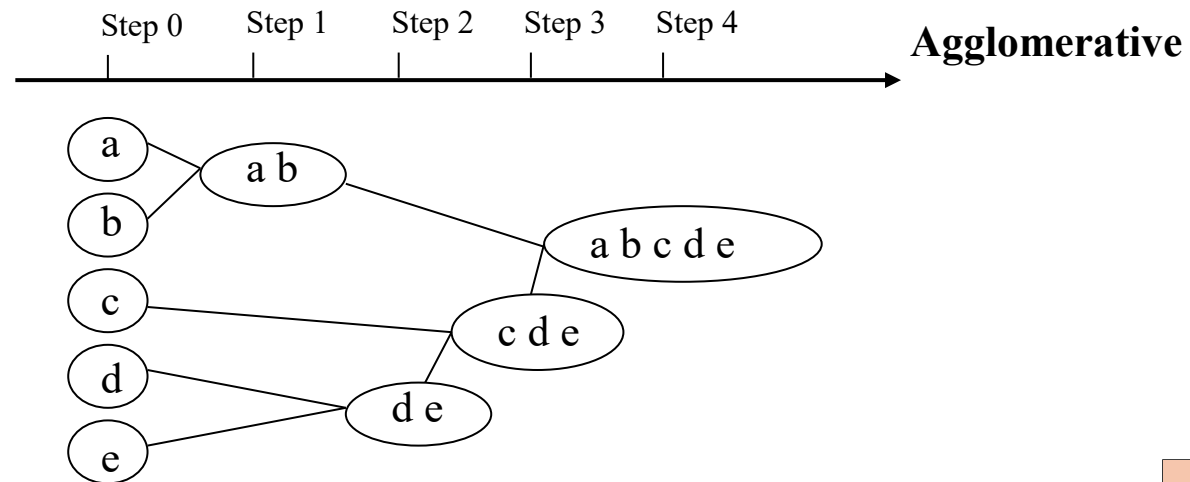
Agglomerative vs. Divisive

- Agglomerative and divisive clustering on the data set {a, b, c, d, e}



Agglomerative vs. Divisive

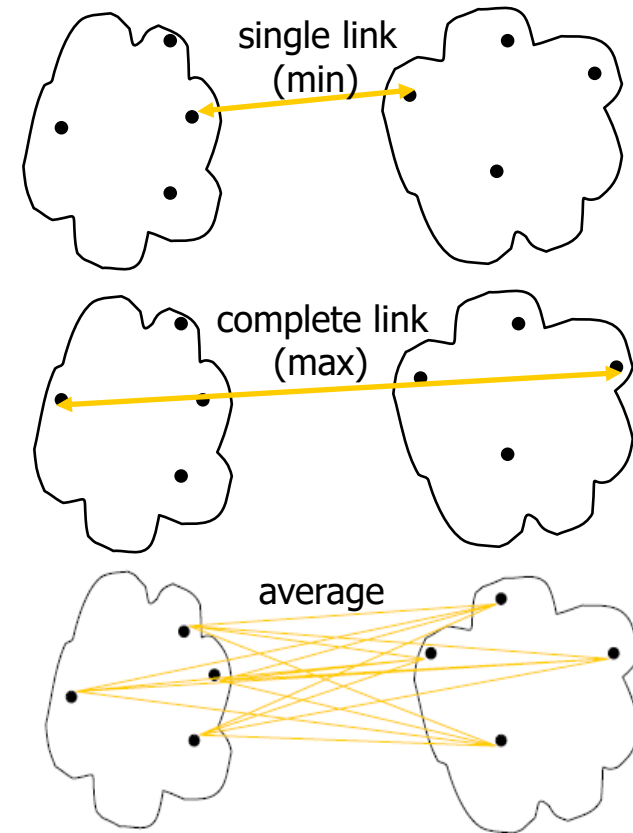
- Agglomerative and divisive clustering on the data set {a, b, c, d, e}



- Cluster distance
- Termination condition

Cluster Distance Measures

- **Single link:** smallest distance between an element in one cluster and an element in the other, i.e., $d(C_i, C_j) = \min\{d(x_{ip}, x_{jq})\}$
- **Complete link:** largest distance between an element in one cluster and an element in the other, i.e., $d(C_i, C_j) = \max\{d(x_{ip}, x_{jq})\}$
- **Average:** avg distance between elements in one cluster and elements in the other, i.e., $d(C_i, C_j) = \text{avg}\{d(x_{ip}, x_{jq})\}$



Cluster Distance Measures

Example: Given a data set of five objects characterized by a single continuous feature, assume that there are two clusters: C1: {a, b} and C2: {c, d, e}.

Objects	a	b	c	d	e
feature	1	2	4	5	6

1. Calculate the distance matrix .

	a	b	c	d	e
a	0	1	3	4	5
b	1	0	2	3	4
c	3	2	0	1	2
d	4	3	1	0	1
e	5	4	2	1	0

Cluster Distance Measures

	a	b	c	d	e
a	0	1	3	4	5
b	1	0	2	3	4
c	3	2	0	1	2
d	4	3	1	0	1
e	5	4	2	1	0

2. Calculate three cluster distances between C1 and C2.

- Single link

$$\begin{aligned}\text{dist}(C_1, C_2) &= \min\{d(a, c), d(a, d), d(a, e), d(b, c), d(b, d), d(b, e)\} \\ &= \min\{3, 4, 5, 2, 3, 4\} = 2\end{aligned}$$

- Complete link

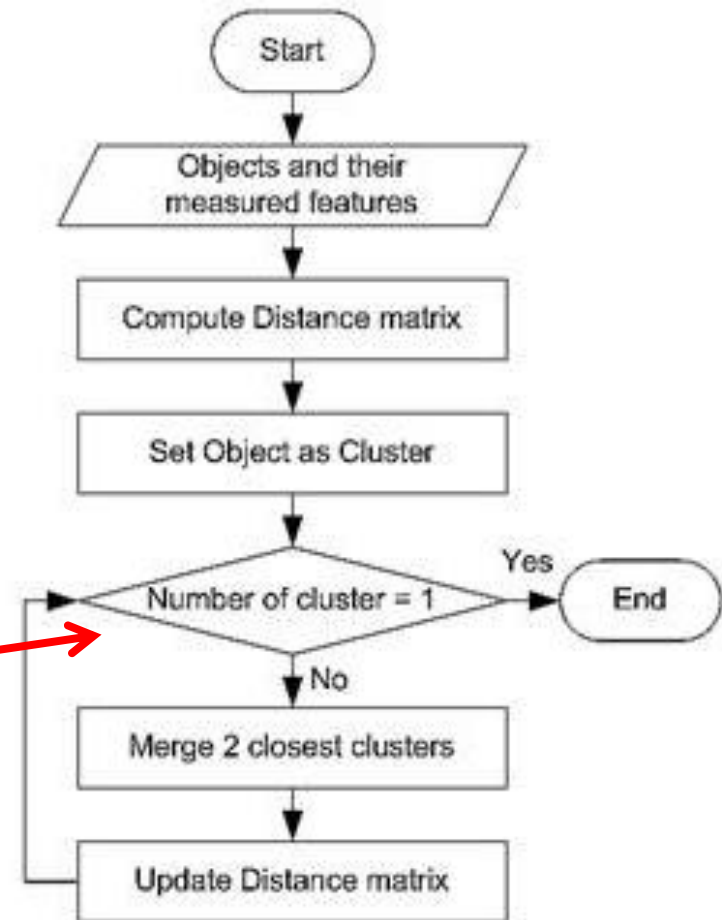
$$\begin{aligned}\text{dist}(C_1, C_2) &= \max\{d(a, c), d(a, d), d(a, e), d(b, c), d(b, d), d(b, e)\} \\ &= \max\{3, 4, 5, 2, 3, 4\} = 5\end{aligned}$$

- Average

$$\begin{aligned}\text{dist}(C_1, C_2) &= \frac{d(a, c) + d(a, d) + d(a, e) + d(b, c) + d(b, d) + d(b, e)}{6} \\ &= \frac{3 + 4 + 5 + 2 + 3 + 4}{6} = \frac{21}{6} = 3.5\end{aligned}$$

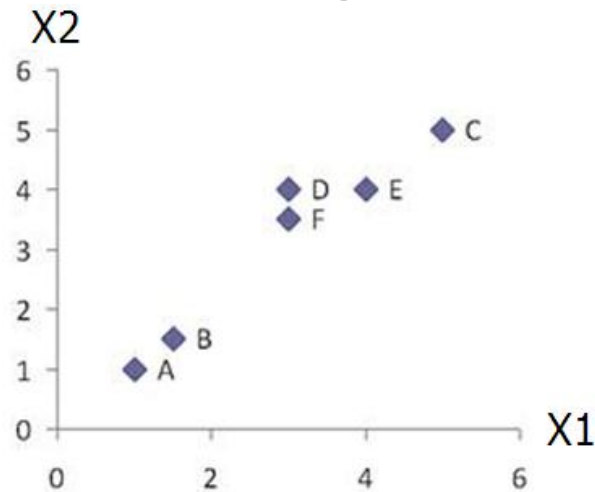
Agglomerative Algorithm

- The Agglomerative algorithm is carried out in three steps:
- Convert all object features into a distance matrix
- Set each object as a cluster (thus if we have N objects, we will have N clusters at the beginning)
- Repeat until number of cluster is one (or known # of clusters)
 - Merge two closest clusters
 - Update “distance matrix”



Agglomerative Algorithm-Example

- Problem: clustering analysis with agglomerative algorithm



$$d_{AB} = \left((1-1.5)^2 + (1-1.5)^2 \right)^{\frac{1}{2}} = \sqrt{\frac{1}{2}} = 0.7071$$

$$d_{DF} = \left((3-3)^2 + (4-3.5)^2 \right)^{\frac{1}{2}} = 0.5$$

Euclidean distance

	X1	X2
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5

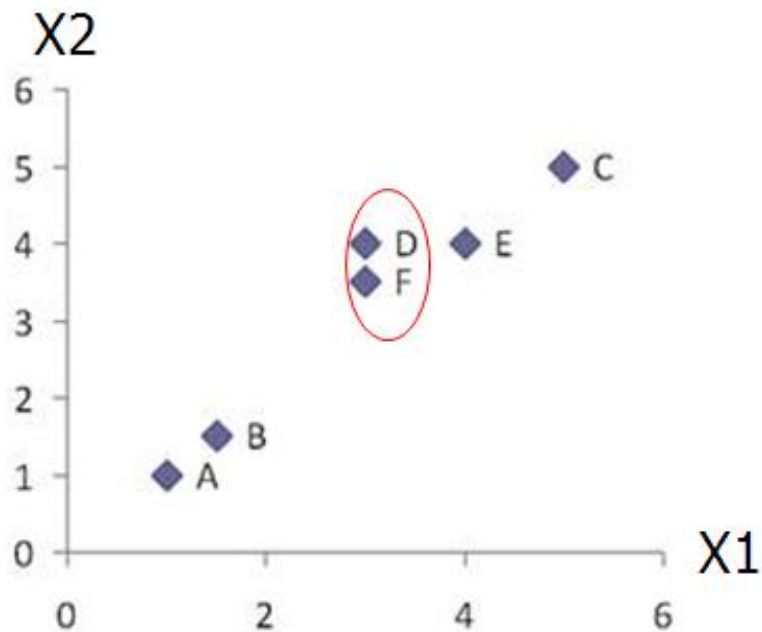
data matrix

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

distance matrix

Agglomerative Algorithm - Example

- Merge two closest clusters (iteration 1)



Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	?	4.24
B	0.71	0.00	4.95	?	3.54
C	5.66	4.95	0.00	?	1.41
D, F	?	?	?	0.00	?
E	4.24	3.54	1.41	?	0.00

Agglomerative Algorithm - Example

- Update distance matrix (iteration 1)

Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00



Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	?	4.24
B	0.71	0.00	4.95	?	3.54
C	5.66	4.95	0.00	?	1.41
D, F	?	?	?	0.00	?
E	4.24	3.54	1.41	?	0.00

Min Distance (single linkage)

$$d_{(D,F) \rightarrow A} = \min(d_{DA}, d_{FA}) = \min(3.61, 3.20) = 3.20$$

$$d_{(D,F) \rightarrow B} = \min(d_{DB}, d_{FB}) = \min(2.92, 2.50) = 2.50$$

$$d_{(D,F) \rightarrow C} = \min(d_{DC}, d_{FC}) = \min(2.24, 2.50) = 2.24$$

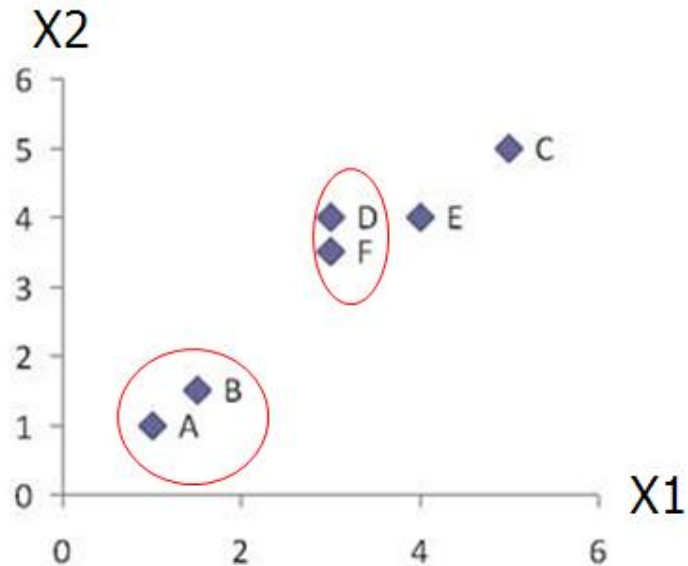
$$d_{E \rightarrow (D,F)} = \min(d_{ED}, d_{EF}) = \min(1.00, 1.12) = 1.00$$



Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00

Agglomerative Algorithm - Example

- Merge two closest clusters (iteration 1)



Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00

Dist	A,B	C	(D, F)	E
A,B	0	?	?	?
C	?	0	2.24	1.41
(D, F)	?	2.24	0	1.00
E	?	1.41	1.00	0

Agglomerative Algorithm - Example

- Update distance matrix (iteration 1)

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00



Dist	A,B	C	(D, F)	E
A,B	0	?	?	?
C	?	0	2.24	1.41
(D, F)	?	2.24	0	1.00
E	?	1.41	1.00	0

Min Distance (single linkage)

$$d_{C \rightarrow (A,B)} = \min(d_{CA}, d_{CB}) = \min(5.66, 4.95) = 4.95$$

$$d_{(D,F) \rightarrow (A,B)} = \min(d_{DA}, d_{DB}, d_{FA}, d_{FB})$$

$$= \min(3.61, 2.92, 3.20, 2.50) = 2.50$$

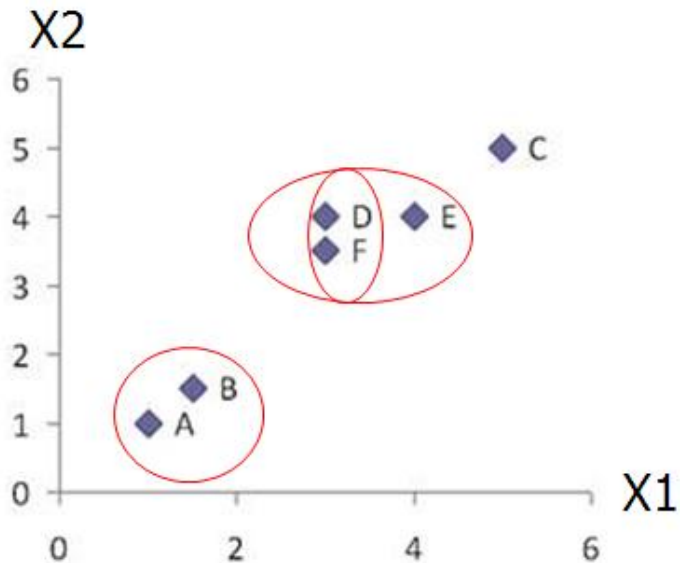


$$d_{E \rightarrow (A,B)} = \min(d_{EA}, d_{EB}) = \min(4.24, 3.54) = 3.54$$

Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	4.95	0	2.24	1.41
(D, F)	2.50	2.24	0	1.00
E	3.54	1.41	1.00	0

Agglomerative Algorithm - Example

- Merge two closest clusters (iteration 1)



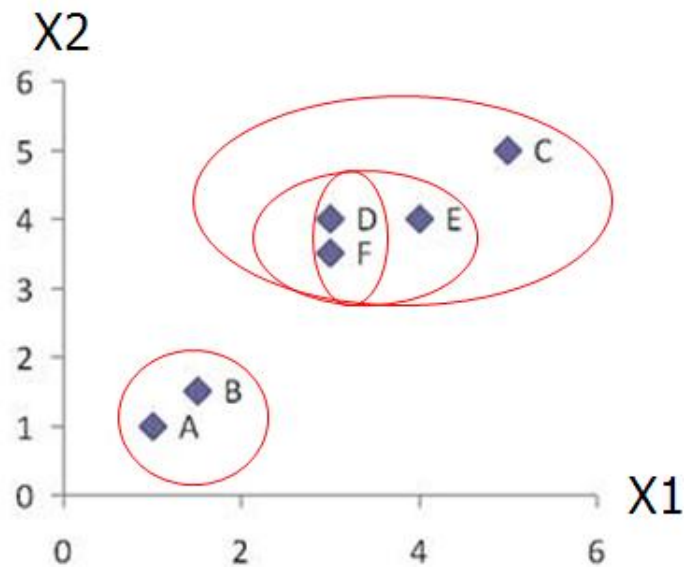
Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	4.95	0	2.24	1.41
(D, F)	2.50	2.24	0	1.00
E	3.54	1.41	1.00	0

Min Distance (Single Linkage)

Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	2.50	1.41	0.00

Agglomerative Algorithm - Example

- Merge two closest clusters (iteration 1)



Min Distance (Single Linkage)

Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	2.50	1.41	0.00

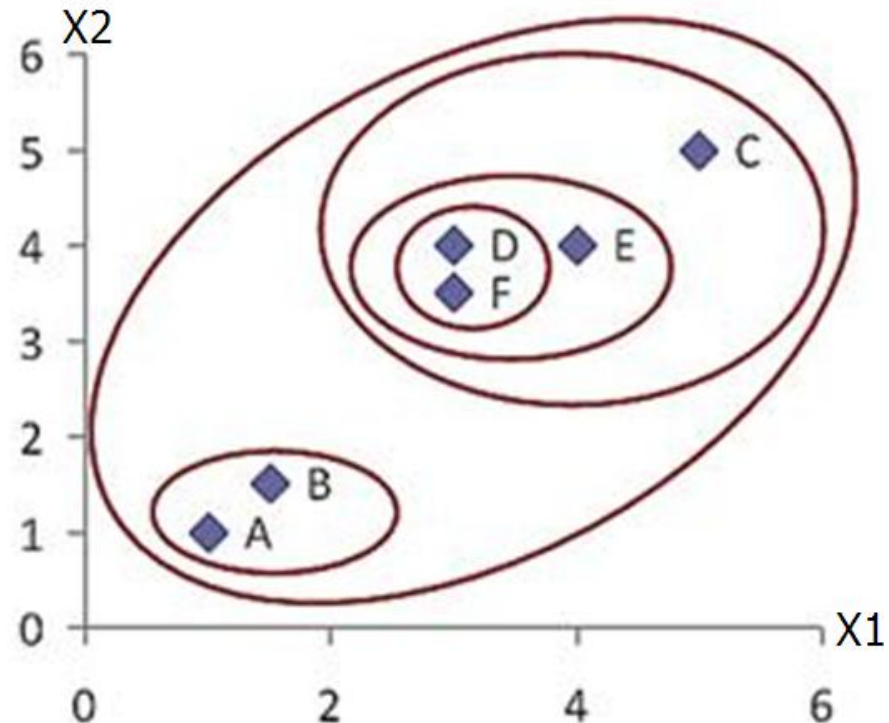
Min Distance (Single Linkage)

Dist	(A,B)	((D, F), E), C
(A,B)	0.00	2.50
((D, F), E), C	2.50	0.00

Agglomerative Algorithm

- Final result (meeting termination condition)

	X1	X2
A	1	1
B	1.5	1.5
C	5	5
D	3	4
E	4	4
F	3	3.5



Dist	A	B	C	D	E	F
A	0.00	0.71	5.66	3.61	4.24	3.20
B	0.71	0.00	4.95	2.92	3.54	2.50
C	5.66	4.95	0.00	2.24	1.41	2.50
D	3.61	2.92	2.24	0.00	1.00	0.50
E	4.24	3.54	1.41	1.00	0.00	1.12
F	3.20	2.50	2.50	0.50	1.12	0.00

Dist	A	B	C	D, F	E
A	0.00	0.71	5.66	3.20	4.24
B	0.71	0.00	4.95	2.50	3.54
C	5.66	4.95	0.00	2.24	1.41
D, F	3.20	2.50	2.24	0.00	1.00
E	4.24	3.54	1.41	1.00	0.00

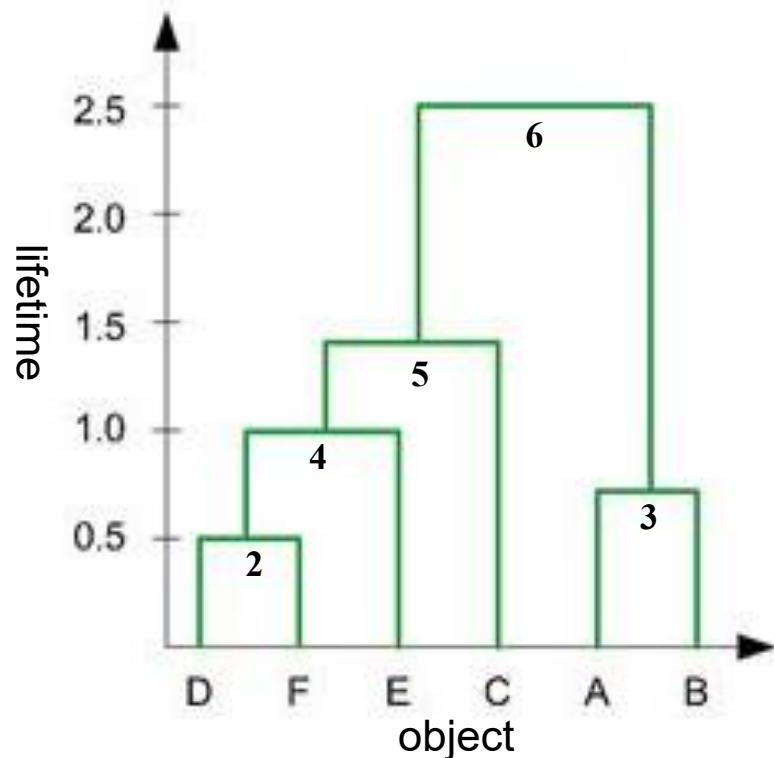
Dist	A,B	C	(D, F)	E
A,B	0	4.95	2.50	3.54
C	4.95	0	2.24	1.41
(D, F)	2.50	2.24	0	1.00
E	3.54	1.41	1.00	0

Dist	(A,B)	C	(D, F), E
(A,B)	0.00	4.95	2.50
C	4.95	0.00	1.41
(D, F), E	2.50	1.41	0.00

Dist	(A,B)	((D, F), E), C
(A,B)	0.00	2.50
((D, F), E), C	2.50	0.00

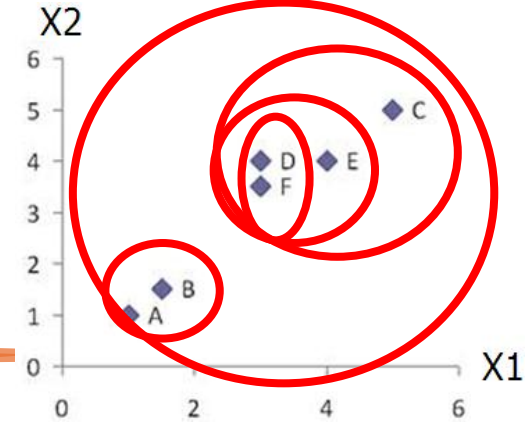
Key Concepts in Hierarchical Clustering

- Dendrogram tree representation



1. In the beginning we have 6 clusters: A, B, C, D, E and F
2. We merge clusters D and F into cluster (D, F) at distance 0.50
3. We merge cluster A and cluster B into (A, B) at distance 0.71
4. We merge clusters E and (D, F) into ((D, F), E) at distance 1.00
5. We merge clusters ((D, F), E) and C into (((D, F), E), C) at distance 1.41
6. We merge clusters (((D, F), E), C) and (A, B) into ((((D, F), E), C), (A, B)) at distance 2.50
7. The last cluster contain all the objects, thus conclude the computation

Lifetime vs K-cluster Lifetime



- **Lifetime**

The distance between that a cluster is created and that it disappears (merges with other clusters during clustering).

e.g. lifetime of A, B, C, D, E and F are 0.71, 0.71, 1.41, 0.50, 1.00 and 0.50, respectively, the life time of (A, B) is $2.50 - 0.71 = 1.79$,

- **K-cluster Lifetime**

The distance from that K clusters emerge to that K clusters vanish (due to the reduction to $K-1$ clusters).

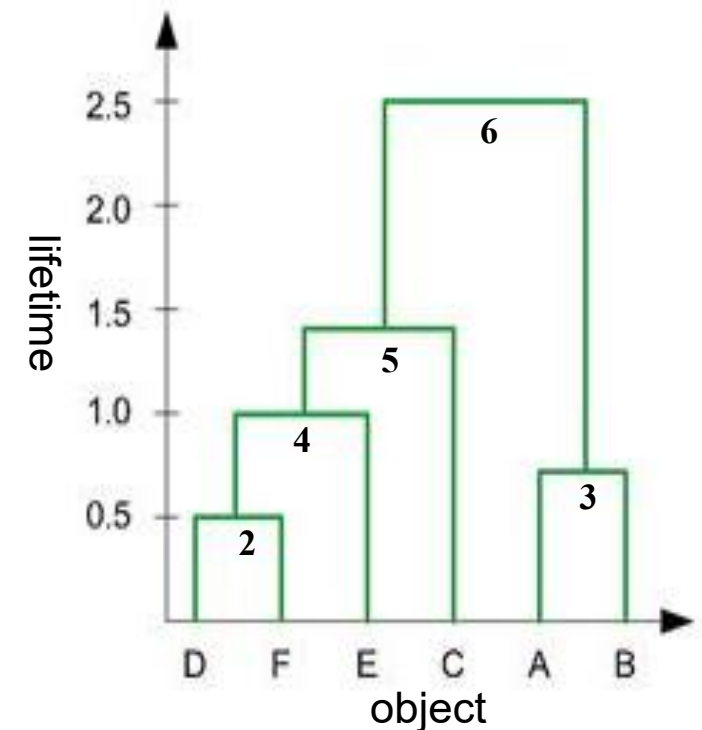
e.g.

5-cluster lifetime is $0.71 - 0.50 = 0.21$

4-cluster lifetime is $1.00 - 0.71 = 0.29$

3-cluster lifetime is $1.41 - 1.00 = 0.41$

2-cluster lifetime is $2.50 - 1.41 = 1.09$

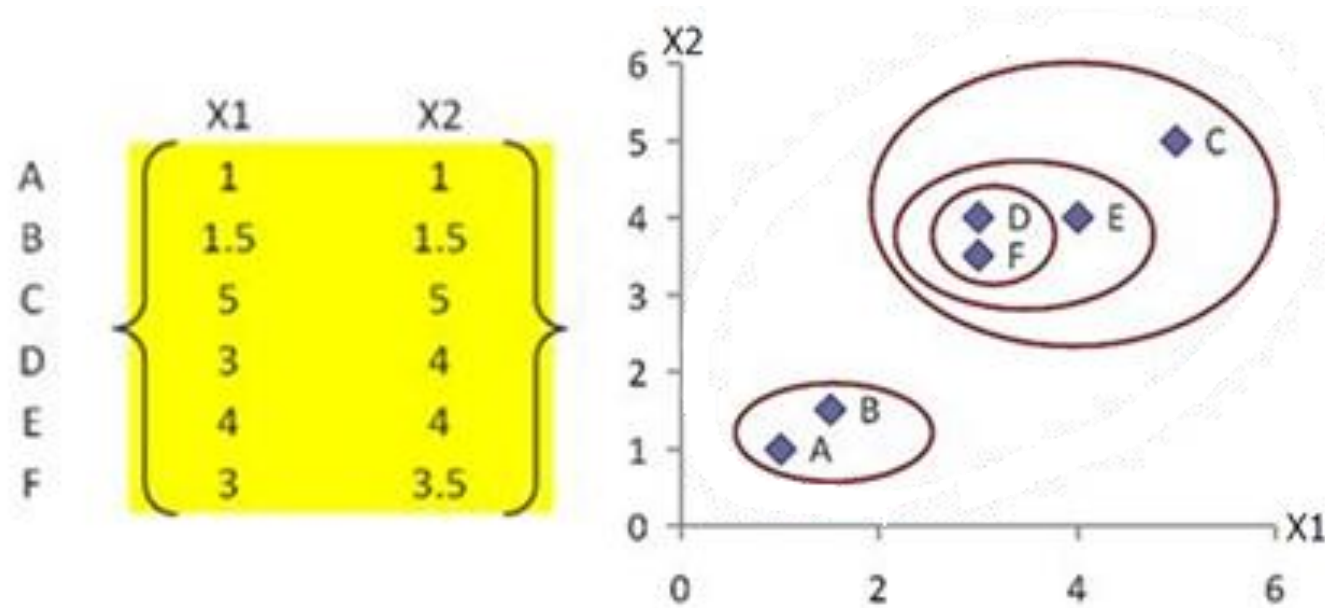


Agglomerative Algorithm-Relevant Issue

- How to determine the number of clusters
 - If the number of clusters known, termination condition is given!
 - The ***K*-cluster lifetime** as **the range of threshold value** on the dendrogram tree that leads to the identification of *K* clusters
 - Heuristic rule: **Cut a dendrogram tree with maximum life time to find a “proper” *K***
- Major weakness of agglomerative clustering methods
 - Can never undo what was done previously
 - Sensitive to cluster distance measures and noise/outliers
 - Less efficient: $O(n^2 \log n)$, where *n* is the number of total objects
- There are several **variants** to overcome its weaknesses
 - **BIRCH**: scalable to a large data set
 - **ROCK**: clustering categorical data
 - **CHAMELEON**: hierarchical clustering using dynamic modelling

Agglomerative Algorithm

- Final result (meeting termination condition)



Agglomerative Algorithm-Summary

- **Hierarchical** algorithm is a sequential clustering algorithm
 - Use distance matrix to construct a tree of clusters (**dendrogram**)
 - Hierarchical representation without the need of knowing # of clusters (can set termination condition with known # of clusters)
- Major weakness of agglomerative clustering methods
 - Can never undo what was done previously
 - Sensitive to cluster distance measures and noise/outliers
 - Less efficient: $O(n^2 \log n)$, where n is the number of total objects

Online tutorial: how to use hierarchical clustering functions in Matlab:
<https://www.youtube.com/watch?v=aYzjenNNOcc>